

CEDAR: Agent-Orchestrated Tree Search for Goal-Directed Optimization of Complex Systems

Yingtao Tian¹

¹Sakana AI, Tokyo, Japan
alantian@sakana.ai

Abstract

Complex systems, core objects of study in artificial life, model diverse phenomena through nonlinear, feedback-driven interactions that produce emergent behavior, with applications from population dynamics and biology to economic policy and strategic decision-making. Yet the difficulty of predicting how feedback structure gives rise to emergent behavior, a central open problem in artificial life, makes goal-directed design exceptionally challenging. In established practice, system structures are written in specialized modeling languages such as DYNAMO or STELLA, compounding the challenge with labor-intensive workflows that limit adoption and hinder timely decision-making.

To address these challenges, we introduce CEDAR, an autonomous method that uses Large Language Model (LLM) agents to discover complex systems satisfying user-specified behavioral goals. Our key innovation is an LLM-driven Monte Carlo Tree Search (MCTS) deeply coupled with complex systems: at each iteration, an LLM Judge evaluates emergent behavior against specified goals and an LLM Editor proposes improved variants, with the Judge acting as a fitness function and the Editor as a variation operator, akin to a generate-and-evaluate loop in evolutionary computation. We represent complex systems as a restricted, runnable subset of Python with domain-specific primitives, letting LLMs modify system dynamics directly. CEDAR formalizes this as an MCTS variant with an LLM-parameterized transition kernel and value function, enabling goal-directed discovery of complex system behaviors while preserving solution diversity, and its LLM-based interpretability reveals how structural changes drive emergent behavior. CEDAR reduces human effort while enabling capabilities difficult to achieve with existing approaches, facilitating broader adoption of complex systems across domains.

Introduction

Computational system modeling provides a framework for studying real-world phenomena as nonlinear, feedback-driven *complex systems*, computational models whose global behavior emerges from feedback interactions among components (Radzicki and Taylor, 2008; Richmond, 1985). Such

systems are central objects of study in artificial life (Bedau, 2003; Langton, 2019; Gershenson, 2023) and systems thinking research (Anderson and Johnson, 1997). This framework has been applied in various domains such as global dynamics (Forrester, 1971), pandemic diffusion (Zhu et al., 2025), social simulations (Kolson, 1996), and biological systems (Hannon and Ruth, 2014), critically supporting policy design and strategic decision-making (Peterson, 2003; Schünemann et al., 2024). Yet predicting how feedback structure gives rise to emergent behavior, a central open problem in artificial life, remains notoriously difficult, making goal-directed design of such systems exceptionally challenging.

A further challenge is practical: established practice often involves specialized modeling languages such as DYNAMO (Radzicki and Taylor, 2008) and STELLA (Richmond, 1985), which require extensive manual effort to navigate unclear feedback relationships (Güneralp, 2004) and labor-intensive workflows (Stermann, 2000), limiting adoption across domains where complex systems modeling could provide valuable insights. Large language models (LLMs) and LLM-powered agents now offer new capabilities for automated discovery of complex systems. In particular, when combined with Monte Carlo Tree Search (MCTS) (Kocsis and Szepesvári, 2006), LLMs show strong capabilities in planning (Zhao et al., 2023; Gao et al., 2024) and reasoning (Song et al., 2025; Chi et al., 2025). This opens promising directions for searching over spaces of complex systems, including artificial life discovery (Kumar et al., 2025; Nisioti et al., 2024), automated scientific discovery (Yamada et al., 2025), and evolving models to generate and falsify hypotheses about biological dynamics (Srinivasan et al., 2025).

We propose CEDAR (Complex-systems Exploration and Design via Agent-Orchestrated Refinement), an autonomous method for discovering complex systems that meet specified goals by combining a unified system representation, the LLM Judge and Editor, and MCTS. At each iteration, MCTS selects a system variant for expansion, an LLM Editor generates improved variants, and the variants are executed and evaluated by an LLM Judge. Concretely, the LLM Editor acts as a variation operator proposing new system structures (Lehman

et al., 2023), while the LLM Judge serves as a fitness function evaluating each candidate, casting system discovery as an evolutionary search process guided by MCTS, where the search tree plays the role of a structured candidate population. This is enabled by our unified representation using a restricted Python subset with domain-specific primitives and inline documentation.

In sum, CEDAR enables goal-directed discovery of complex system behaviors with capabilities difficult to achieve with existing approaches, while preserving solution diversity that supports open-ended exploration, demonstrates applicability on classical systems drawn from social and biological modeling, and makes the emergent mechanisms of complex systems more transparent through LLM-based interpretability, lowering the barrier to constructing and studying complex systems in artificial life and beyond.

Related Work

Complex Systems and How to Model Them *World Dynamics* (Forrester, 1971) simulates population, resources, and industrial output over long horizons of the planet. This introduces a computational framework for modeling a subject as a nonlinear, feedback-driven system (“complex system”), a paradigm shift (Forrester, 2011) from linear prediction-based modeling. With the help of DYNAMO (Radzicki and Taylor, 2008) and STELLA (Richmond, 1985) modeling languages (online examples of DYNAMO and STELLA show their old syntax and proprietary visual interfaces), complex systems have seen a wide range of applications, including social simulations (Kolson, 1996), war games, pandemic diffusion (Zhu et al., 2025), economics, thermodynamics, and biological systems (Ruth and Hannon, 2012; Hannon and Ruth, 2014). This systems thinking (Anderson and Johnson, 1997; Arnold and Wade, 2015) supports policy design and strategic decision-making (Peterson, 2003; Schünemann et al., 2024). Despite these advances, modeling still requires expert-defined variables and relationships (Ford and Sterman, 1998; Bérard, 2010; Zagonel, 2002): the relationship between structure and emergent behavior is often unclear (Schoenberg et al., 2020; Güneralp, 2004; Barlas, 1996), and established languages require labor-intensive workflows (MIT System Dynamics in Education Project, 1998; Hines, 1996; Sterman, 2000).

Tree Search Algorithms with LLMs Monte Carlo Tree Search (MCTS) (Kocsis and Szepesvári, 2006) has shown promising results when combined with powerful models. MCTS with neural networks has led to superhuman performance in games (Silver et al., 2016, 2017b,a; Schrittwieser et al., 2020), plus the discovery of novel algorithms (Fawzi et al., 2022; Mankowitz et al., 2023) and neural network architectures (Nasir et al., 2024). More recently, MCTS has been combined with large language models (LLMs), including LLM-powered value functions and self-reflection (Zhou et al., 2024), LLM-powered node selection and expansion

strategies (Inoue et al., 2025), as well as applications to specific tasks such as automatic design (Zheng et al., 2025) and agent collaboration (Gan et al., 2025). Notably, MCTS with LLMs offers strong capabilities in planning, where LLMs serve as common-sense policy priors (Zhao et al., 2023) and enhance interpretability (Gao et al., 2024), and in reasoning, where LLMs guide traversals of knowledge graphs and refine reasoning through steps (Song et al., 2025; Chi et al., 2025).

LLMs as Evolutionary Operators in Artificial Life

LLMs have been proposed as variation operators in evolutionary search, replacing random mutation with language-guided program edits (Lehman et al., 2023; Nisioti et al., 2024). A related line evolves language prompts to steer the behavior of simulated cellular and biological systems (Le et al., 2025, 2026). This connects LLMs to a central ALife goal: open-ended discovery of emergent behaviors. Our work extends this to complex systems, with the LLM Editor as variation operator and the LLM Judge as fitness function.

LLM-based Optimization for Structured Systems

LLMs with MCTS show promising results for complex and challenging searches, including automated scientific discovery (Yamada et al., 2025), artificial life discovery (Kumar et al., 2025) and runnable code for reasoning (Katz et al., 2024). Recent works also apply LLMs specifically to model system dynamics (Liu et al., 2024; Luo et al., 2025; Liu and Keith, 2025) by treating models as predictors of behaviors that are *unknown* to the optimizer. These studies consider only smaller systems (up to 4 variables and 12 steps), whereas our method scales to 60 variables and 4000 steps, making direct extensions of them to our scenario non-trivial.

Prior Works Close to Our Method Several prior works combine LLMs with search but target different goals. I-MCTS (Liang et al., 2025) targets AutoML hyperparameters and LATS (Zhou et al., 2024) operates on discrete task graphs, and neither addresses iterative temporal dynamics optimization. GIF-MCTS (Dainese et al., 2024) and LLM-SRBench (Shojaee et al., 2025) address short sequences, whereas our method optimizes multi-thousand-step temporal dynamics. Prompt optimization methods (Tong et al., 2025; Fernando et al., 2024; Khattab et al., 2024) are applicable in principle but require demonstrations unavailable in our online-exploration setting.

Method

Preliminaries

We first formalize the complex systems under discussion in this work. Complex systems model a subject as a nonlinear, feedback-driven mathematical system whose dynamics give rise to emergent behavior. While such systems admit a range of formalisms, including agent-based models, cellular

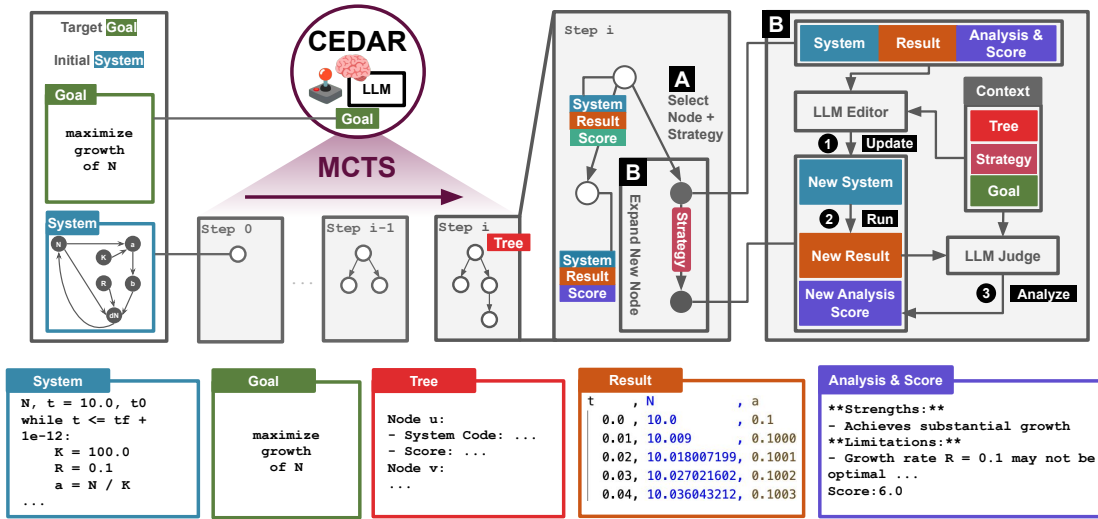


Figure 1: Overall architecture of CEDAR. The autonomous approach uses Monte Carlo Tree Search (MCTS) to iteratively select system variants for expansion. At each iteration, an LLM Editor generates improved system variants and executes them, then an LLM Judge evaluates their performance and provides detailed analysis. The method integrates several components: a system in a restricted subset of Python code, a natural-language goal description, a search tree structure, system execution results, and comprehensive analysis feedback.

automata, and discrete-time dynamic graphs, one common formalism, which we adopt throughout this work, is ordinary differential equations (ODEs) with initial condition: $dx/dt = f(x, t)$, $x(t_0) = x_0$, where $x \in \mathbb{R}^n$ is the state vector (also called level or stock variables), $f: \mathbb{R}^n \times \mathbb{R} \rightarrow \mathbb{R}^n$ the vector field, and $x_0 = [x_{0,1}, \dots, x_{0,n}]^T$ the initial values at t_0 , the initial time. As a result, when f follows a known, simple form, analytical techniques for ODEs (Walter, 2013), such as the Laplace transform (McLachlan, 2014), can be applied to complex systems analysis.

However, in practice, complex systems are more commonly analyzed through numerical approaches like the first-order Euler method (Ogata, 2004) due to the complexity of f when nonlinear feedbacks are present. This approach enables flexible modeling by requiring only that f be computable, which is crucial since many complex systems admit no analytical solution. Concretely, the first-order Euler method discretizes the system with time step Δt as: $x_{t+\Delta t} = x_t + \Delta t \cdot f(x_t, t)$, $x_0 = x(0)$. Note that x and its discretized counterparts x_t and $x_{t+\Delta t}$ represent interpretable, meaningful quantities such as populations and resources, rather than abstract hidden states as in RNNs, despite the similarity in their formulations. Historically, communities studying complex systems have benefited from specialized modeling languages like DYNAMO and STELLA (Radzicki and Taylor, 2008; Richmond, 1985). However, these tools suffer from outdated syntax or proprietary visual interfaces (Forrester, 2011) that limit their usability with LLMs.

Unified Representation

To bridge complex systems with LLMs, we propose a representation of complex systems based on a restricted subset of Python programs, unifying the representations traditionally used in DYNAMO and STELLA. Python is widely familiar, and LLMs have strong Python coding capabilities. This avoids creating a new domain-specific language, and enables flexible implementation of $f(x, t)$ through general-purpose programming. We show an example of this representation in Figure 2.

Besides plain code, we enrich it with documentation and predefined, domain-specific mathematical primitives that aid LLMs in using the representation. Doing so helps LLMs understand the complex system’s semantics, manipulate system dynamics through semantic constructs rather than low-level implementation details, and know what restrictions must be enforced.

Proposed Method

We propose CEDAR, an autonomous method that combines MCTS, LLM, and the unified representation above to discover complex systems that meet a given goal. We present the overall architecture and components of CEDAR in Figure 1 and provide details as follows.

Overview and Formalization. CEDAR optimizes complex systems by editing them towards a goal G , which is a natural-language description of the desired system behavior (for example, “grow population stably”). A system

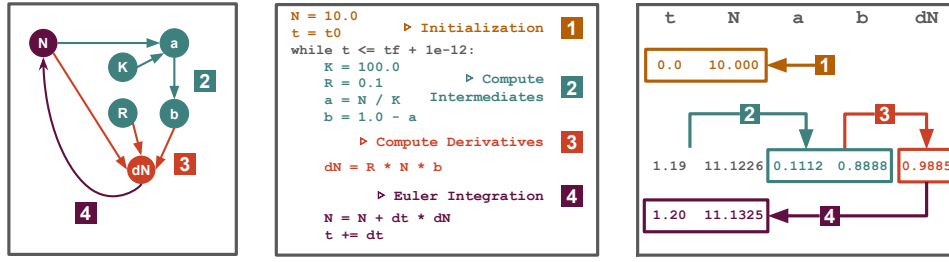


Figure 2: Python representation of a complex system. From left to right: graph representation (for reference), Python code, and computation process. Four essential components are shown: (1) initialization of values ($x_0 = x(0)$), a loop of (2) intermediate computations, (3) derivative evaluations (i.e., computing $f(x_t, t)$), and (4) Euler-method integration (i.e., computing $x_{t+\Delta t}$ from x_t and $f(x_t, t)$). This Python-based representation allows users to easily code and inspect, while leveraging LLM’s coding capabilities.

P is represented using the restricted subset of Python programs. The MCTS iteratively builds $T = (V, E)$ with node set V and edge set E , each time expanding a selected node v and adding new child node u . We denote the expansion strategy used to generate u as s_u , and u ’s parent as $p_u \triangleq v$ (the root node $0 \in V$ has no parent p_0 or expansion strategy s_0). Each node $u \in V$ stores a system P_u , an execution record R_u , a score S_u , and a textual analysis A_u , which are produced in the following way: An LLM Editor produces a new program P_u by patching P_v (e.g. modifying equations, changing feedback links). P_u is then executed to obtain a new execution record $R_u \leftarrow \text{RUN}(P_u)$ which includes values of all variables at each timestamp. The LLM Judge evaluates the quality of system P_u against the specified goal G , producing (A_u, S_u) : $S_u \in \mathbb{R}$ is a bounded numerical score reflecting how well P_u satisfies goal G , and A_u is a textual analysis providing qualitative feedback about the system’s behavior and potential improvements.

Initialization. CEDAR begins by initializing the MCTS tree T with a single root node v_0 , associated with an initial complex system P_0 which can be either provided by the user or generated from a basic template, P_0 ’s execution record $R_0 \leftarrow \text{RUN}(P_0)$ and the evaluation of P_0 : $(A_0, S_0) \leftarrow \text{LLMJUDGE}(P_0, R_0, G)$.

Node Selection. The node selection process jointly selects (v, s_u) , a node $v \in V$ to expand from and an expansion strategy s_u to use, for generating new node u later. Selection of expansion strategy s_u is simply done by uniformly sampling $s_u \sim \text{Uniform}(\mathcal{S})$ where $\mathcal{S} = \{s_1, s_2, \dots, s_k\}$ is a fixed set of expansion strategies. Selection of v is based on $\text{SCORE}(v)$ that combines the node’s performance score S_v , its depth $\text{DEPTH}(v)$ in the tree, its expansion count $c_v \in \mathbb{N}$, and its parent p ’s expansion count c_p (we omit the subscript v and write $p = p_v$): CEDAR uses the generalized UCT score $\text{SCORE}(v) = S_v + \phi(c_v, c_p, \tau)$. Here, we have $\phi(\cdot) = -\infty$ if $c_v \geq \tau$ and $\alpha \cdot \sqrt{\ln(c_p + 1)/(c_v + 1)} + \gamma \cdot \text{DEPTH}(v)$

otherwise, where α adjusts the amount of exploration, $\gamma > 0$ controls the preference for deeper nodes, and τ controls the expansion count. Inspired by Inoue et al. (2025) and unlike vanilla MCTS, we limit expandable nodes to a subset $V_{\text{exp}} \subseteq V$ and allow repeated expansion of internal nodes until their expansion count c_v reaches a threshold $\tau \in \mathbb{N}$, so a node v is expandable if and only if $v \in V_{\text{exp}}$ and $c_v < \tau$. The process selects $v = \arg \max_{v \in V_{\text{exp}}} \text{SCORE}(v)$.

Node Expansion with LLMs. From a selected node v and expansion strategy s_u , the LLM Editor generates a new system variant $P_u \leftarrow \text{LLMEDITOR}(P_v, A_v, s, G)$. Using the system P_v , its associated analysis A_v , the chosen strategy s , and the goal specification G , this expansion produces a revised system that satisfies syntactic constraints and semantic requirements while being more aligned with the goal. Because the system is represented as code, the Editor can make structural edits to P_v rather than only tuning coefficients: it may add or remove state variables, introduce or rewire feedback relations, and change the equations. The newly generated system P_u is executed to obtain its record $R_u \leftarrow \text{RUN}(P_u)$, followed by the evaluation $(A_u, S_u) \leftarrow \text{LLMJUDGE}(P_u, R_u, G)$ that produces a score S_u and a qualitative analysis A_u .

Summary. This iterative process of selection, expansion, execution, and evaluation enables progressive improvement toward the target objectives by exploring the space of complex systems. We formalize the complete search procedure in Algorithm 1 and illustrate particularly the expansion process in Figure 3. Both LLMEDITOR and LLMJUDGE are provided with well-crafted prompts covering system context, tree context, and structural constraints, which alongside code-based representation are essential for the method to work.

Theoretical Connection

While designed empirically, CEDAR is a principled MCTS variant. Node selection is a generalized UCT combining pro-

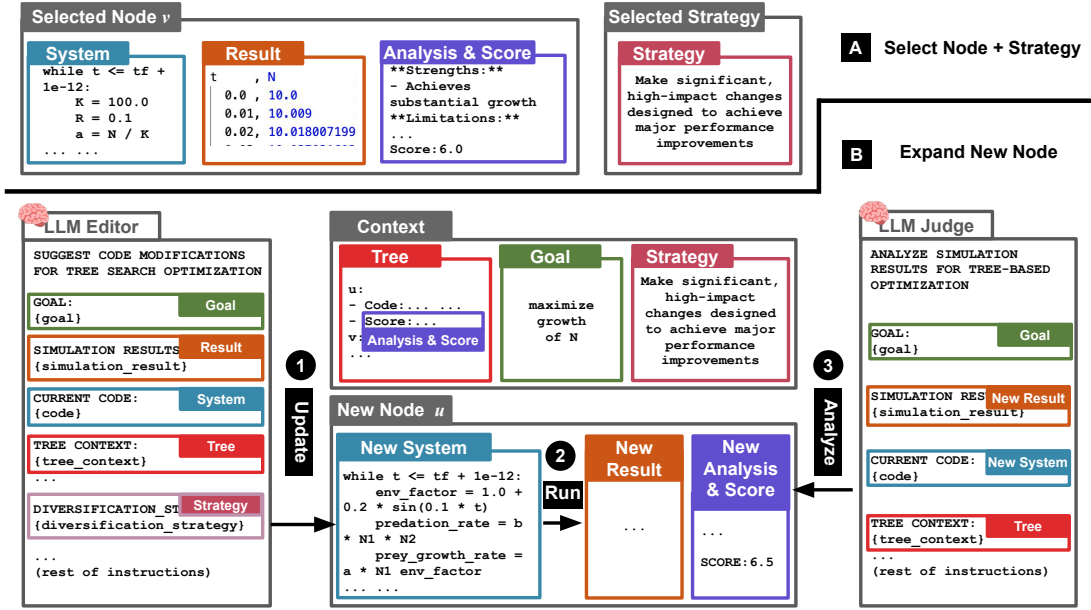


Figure 3: The node expansion process using LLM Editor and Judge. Given a selected node v and expansion strategy s , the LLM Editor generates a new system variant P_u by leveraging the parent system P_v , its analysis A_v , strategy s , and goal G . The generated system is executed to obtain record R_u , then evaluated by the LLM Judge to produce analysis-score pair (A_u, S_u) . We embed analysis and scores of all nodes, including the currently selected node, in the tree context to provide the LLM with a comprehensive view of the search tree.

gressive widening (Chaslot et al., 2008; Inoue et al., 2025) and depth-based bonuses (Blackshaw et al., 2025; Wu et al., 2025): $\text{SCORE}(v) = S_v + \alpha \sqrt{\ln(c_p+1)/(c_v+1)} \cdot \mathbb{I}[c_v < \tau] + \gamma \cdot \text{DEPTH}(v)$. The LLM Editor acts as a stochastic transition kernel over programs, $P_u \sim P_\theta(\cdot | P_v, A_v, s, G)$, connecting to learned-transition MCTS and model-based RL (Silver et al., 2017b; Schrittwieser et al., 2020). The LLM Judge acts as a learned value function providing bounded noisy rewards, connecting to MCTS with biased evaluators (Lisy et al., 2013; Efroni et al., 2019). That said, we note that the convergence and regret guarantees for MCTS with LLMs remain open problems despite strong empirical performances (Brandfonbrener et al., 2024; Katz et al., 2024).

Experiments

CEDAR enables three capabilities difficult for existing approaches: optimizing for vague natural-language goals, fitting records without complete system skeletons, and interpretable optimization. We provide qualitative demonstrations and quantitative comparisons for these capabilities.

Experimental Details

Data. We collect complex systems from classical works: *World Dynamics* (Forrester, 1971) in the DYNAMO language and 19 systems from the book *Modeling Dynamic Biological Systems* (Hannon and Ruth, 2014) in the STELLA language (20 systems, 20 to 69 integrated variables each), converted

Algorithm 1 Monte Carlo Tree Search for Complex System Discovery

- Input:** Initial system P_0 , goal G , iterations N , expansion threshold τ
- Output:** Best discovered system P^*
- Initialize: $T = (V, E)$ with root node 0, $(A_0, S_0) \leftarrow \text{LLMJUDGE}(P_0, \text{RUN}(P_0), G)$, $P^* \leftarrow P_0$, $S^* \leftarrow S_0$
- for** $i = 1$ to N **do**
- $v \leftarrow \arg \max_{v \in V_{\text{exp}}} \text{SCORE}(v)$ $\triangleright$ Select node to expand
- $s \sim \text{Uniform}(S)$ $\triangleright$ Sample expansion strategy
- $P_u \leftarrow \text{LLMEDITOR}(P_v, A_v, s, G)$ $\triangleright$ Expand
- $R_u \leftarrow \text{RUN}(P_u)$ $\triangleright$ Execute
- $(A_u, S_u) \leftarrow \text{LLMJUDGE}(P_u, R_u, G)$ $\triangleright$ Evaluate
- Add node u to the tree with (P_u, R_u, A_u, S_u)
- $c_v \leftarrow c_v + 1$ $\triangleright$ Increment expansion count
- if** $c_v \geq \tau$ **then**
- $V_{\text{exp}} \leftarrow V_{\text{exp}} \setminus \{v\}$ $\triangleright$ Drop from expandable set
- end if**
- if** $S_u > S^*$ **then**
- $P^* \leftarrow P_u$, $S^* \leftarrow S_u$
- end if**
- end for**
- return** P^*

into our representation. We utilize these systems in two ways: (1) as given models to be further optimized towards multiple goals, and (2) as ground truth systems that produce reference records serving as optimization targets.

Setup. In our experiment, we conduct 20 node selections, during each of which we expand to 5 new candidate nodes. This results in a total of 100 expansions per search. We empirically set hyperparameters $\alpha = 1$ and $\gamma = 2$ for node selection. The expansion strategy is selected by sampling from a set of candidate strategies listed in Table 1.

Table 1: MCTS expansion strategies.

Strategy	LLM Instructions
breakthrough	Make significant, high-impact changes for major performance improvements
aggressive	Make bold structural or algorithmic changes
amplify	Identify and significantly increase the most promising parameters
exploratory	Try completely different parameter combinations
targeted	Focus on specific high-impact parameters from analysis
contrarian	Try approaches opposite to current trends
balanced	Make moderate parameter changes with good risk/reward ratio
conservative	Make small, incremental parameter adjustments

We use adaptive context subsampling for scalability, keeping the execution record within a context budget L and achieving $O(NL)$ overall complexity. Because the LLM Editor and Judge define the transition and evaluation operators, we treat their settings as part of the method: all runs use two strong-coding providers, Anthropic Claude Sonnet 4.5 and GPT-5.1, at default decoding settings with structured outputs and up to three retries, and assign crashed or NaN-valued runs a low score so MCTS naturally backtracks. Our runs incur moderate computational costs and runtime (\$10–\$50, 30 minutes per run).

Fitting an Abstract Goal Described in Natural Language

In this experiment, we task CEDAR with optimizing a given good-enough complex system towards a further ambitious goal. The starting system is *World Dynamics* (Forrester, 1971), a published, hand-designed model of the co-evolution of human population, resource utilization, and pollution, containing the most variables in our data and posing challenges for understanding the system dynamics. Our natural-language goal is to further optimize it to achieve more population growth, less resource usage, and less pollution, all *jointly*:

Natural-language goal

Balance population, resources, and environment by optimizing to (1) maximize population growth, (2) minimize the resource depletion rate, and (3) minimize the pollution accumulation rate. Seek the best trade-off where the population grows sustainably without depleting resources too quickly or creating excessive pollution. (Important: only change the coefficients in the helper. Do not change any coefficient by more than \$50\\%\$ to prevent variable explosion.)

Doing so poses intrinsic challenges: the hand-designed system already reaches a kind of Pareto frontier, where optimizing one sub-goal often comes at the expense of others, as the subgoals are conflicting in nature, as Figure 4 shows.

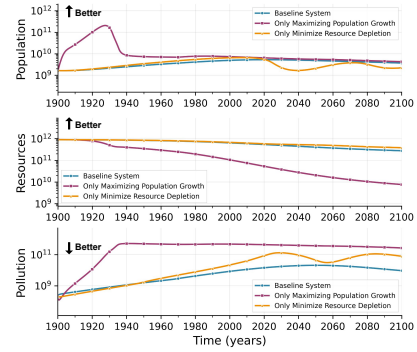


Figure 4: Optimizing for a single goal at a time: focusing on one objective hurts the others, showing competitive subgoals. As in Figure 5, the Resources panel plots the *remaining* resource stock, so its upward “better” direction corresponds to less depletion.

Figure 5 illustrates the MCTS search tree and the resulting system dynamics, and shows how the tree structure guides the search toward higher-scoring solutions. The resulting system satisfies the vague goal, with all three key variables (population, resource, pollution) improving over the initial system. Interestingly, with different LLM backends, CEDAR discovers multiple high-performing systems that are structurally distinct, each emphasizing different tradeoffs among objectives (for example, prioritizing population by Claude versus resource by GPT-5.1).

Quantitative Studies: Fitting a Concrete Record

In this experiment, we task CEDAR with evolving a system (using only a bare-minimum skeleton) to fit the simulated record of a target complex system. While the strength of CEDAR lies in fitting vague, abstract goals, it can nonetheless be used to fit records from either observations or ground truth systems, by simply loading the record into the goal and running CEDAR as-is. The ground truth complex system is

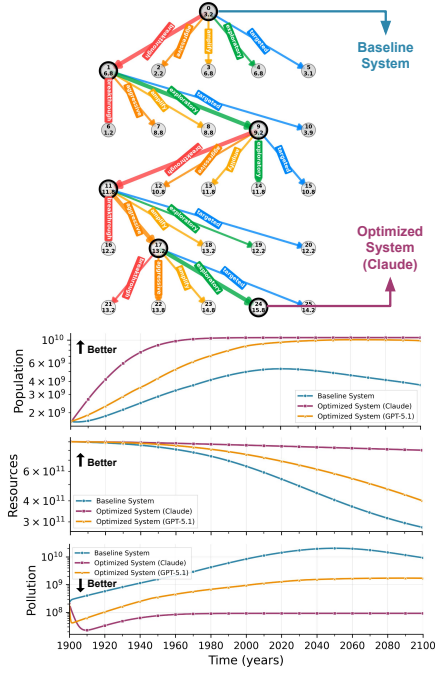


Figure 5: Illustration of fitting an abstract, language-based goal from *World Dynamics*. Top: The MCTS tree with the path to best node highlighted. Each node v is marked by its index and LLM Judge’s score S_v . Bottom: The system dynamics for the initial (original) and final complex systems with two LLM backends. We show key variables including Population ($\uparrow$), Pollution ($\downarrow$), and Resources ($\uparrow$). Here Resources denotes the *remaining* natural-resource stock, so a higher value reflects less depletion and is preferable. With CEDAR, the system reaches better states (increased population, less resource depletion, and less pollution).

a population growth model with stochastic components (a stochastic death rate sampled at each time step, introducing volatility intrinsic to the system). We compare CEDAR with existing approaches based on Optuna (Akiba et al., 2019) black-box optimization. Since the search space for black-box optimization consists of only coefficients, we explore three levels of formula support: no formulae, a simple population-dependent death rate, or the full ground truth formulae, the last of which gives the baseline a serious boost by reducing the task to parameter fitting. We note this comparison deliberately favors the baseline: each Optuna variant is granted 100 trials and, in its strongest setting, the full ground-truth formulae, which is strictly more prior structure than CEDAR receives, as CEDAR starts from a bare-minimum skeleton. To measure the accuracy of fitting a concrete record, we employ both L1 and Dynamic Time Warping (DTW) (Sakoe and Chiba, 1978) distances as metrics. DTW measures trajectory similarity under optimal temporal alignment using a Sakoe-Chiba band ($w=250$, 7.1% of the 3500-step sequence).

Table 2: L1 and DTW distances to the target record (lower is better). The Formulae columns indicate how much of the ground-truth structure each method is given: No (no formulae), S. (a simple population-dependent death rate), or F. (the full ground-truth formulae). CEDAR, given no formulae, achieves lower error than the Optuna baseline given the full formulae.

Method	Formulae			L1	DTW
	No	S.	F.		
Optuna	✓			29.06	5503.68
Optuna		✓		26.01	4927.94
Optuna			✓	3.71	477.52
CEDAR (Claude)	✓			3.29	757.21
CEDAR (GPT-5.1)	✓			2.22	433.13

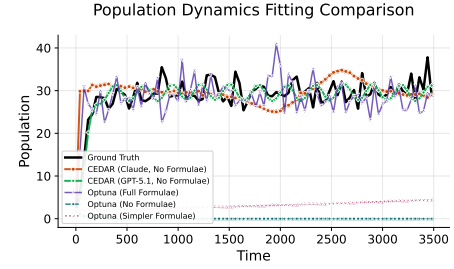


Figure 6: System dynamics (population) when fitting a concrete record. CEDAR, without predefined formulae, tracks the ground-truth trajectory closely, while the no- and simple-formulae Optuna baselines collapse to near-constant populations; only Optuna with the full formulae stays competitive. (All trajectories rise together during the early ramp-up, where the legend overlaps the curves.)

As shown in Table 2 and Figure 6, the Optuna baselines are severely limited by the skeletal formulae: the no/simple-formulae variants are clearly worse, quantitatively and qualitatively, than the full-formulae variant. Meanwhile, CEDAR, even without predefined formulations, finds solutions that outperform Optuna with a full formula skeleton. Across 10 seeds, the ground truth’s stochasticity (a death rate drawn from a normal distribution each step, scaled by population) shows volatility is intrinsic to the system, not an optimizer artifact; CEDAR’s record generally falls within the ground-truth variability and captures the magnitude and timing of major peaks and dips. Furthermore, even with full-formulae support and 100 trials, Optuna does not perfectly match the dynamics (L1 3.71–4.26 across two runs), as the stochastic noise floor and high-dimensional nonlinear parameter space make purely numeric search insufficient.

Interpretability

For the *World Dynamics* system above, we examine the interpretability of CEDAR through the LLM’s analysis of candidate systems, highlighting in Table 3 the LLM responses along the path that produces the best system in the search tree of Figure 5. CEDAR exposes clear, human-readable analyses at each step that document how the three subgoals are balanced. The LLM Judge analyzes issues with balanced focus on subgoals, while the LLM Editor first evaluates overall performance, then fine-tunes individual subgoals, also attending to unmentioned aspects (e.g., capital productivity). These analyses are the stated rationale accompanying each edit, aiding inspection of the search; they are not a verified causal account of how structure produces emergent behavior, which remains an open problem.

Table 3: LLM Editor and Judge responses along the best path in the tree. We show the LLM-summarized version with manually colored **Population**, **pollution**, and **resources**.

LLM Editor Response	LLM Judge Response
(No editor response for root, which is the initial system given as is.)	Unsustainable overshoot behavior - 69% resource depletion , 46x pollution increase
Breakthrough approach - 25-40% efficiency improvements targeting 6-8 score	Sustainable progress - population to 5.46B, 30% resources remaining , controlled pollution
Exploratory approach - radical efficiency through enhanced capital productivity	Paradigm shift - 7.0B population , 78% resource conservation , near-zero pollution
Ultra-aggressive conservation - pushing toward exceptional 9.5+ performance	Breakthrough sustainability - 3.2x population , 36% resource depletion , 90% pollution improvement
Revolutionary breakthrough - 75% resource reduction , 90% pollution control , 300% capital productivity	Best-in-tree performance - 6.3x population , 59% resource conservation , exceptional pollution control
Extreme efficiency breakthrough - 95% resource conservation , 98% pollution reduction , 15x capital productivity	Holy grail achievement - 6.4x population growth with only 19.9% resource depletion

Why Use MCTS with LLMs: Diversity of Solutions and Better Performance

We find the benefits of MCTS to be threefold. **First, MCTS helps preserve solution diversity.** For the record-fitting experiment, we sample nodes within a single run whose scores are close to the best solution and visualize their systems’ populations in Figure 7. All satisfy the goal, yet follow distinct trajectories: unlike the Optuna baselines, CEDAR avoids collapsing to a single overfitting-like point, which enables better decision-making through sensitivity analysis.

Second, MCTS leads to better performance. Our ablation against linear search (Figure 8) shows MCTS yields higher-scoring nodes and smoother trajectories that avoid

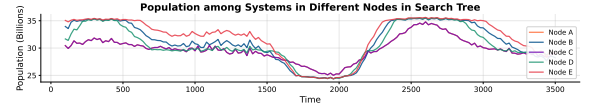


Figure 7: Diversity of solutions in the search tree showing different population dynamics after systems pass the initial phase ($t > 100$).

overfitting-like behavior, indicating MCTS with LLMs is crucial for performance gains.

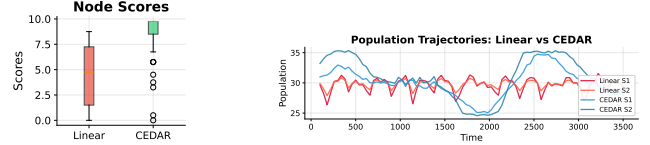


Figure 8: MCTS versus linear search. Left: MCTS attains higher-scoring nodes than linear search. Right: the resulting population dynamics are smoother and avoid the overfitting-like behavior of linear search, showing that tree-based exploration drives CEDAR’s performance.

Third, MCTS supports open-ended discovery of structurally distinct solutions. Because MCTS builds a branching tree rather than converging to a single point, it preserves diversity, which matters in ALife contexts where exploring a range of emergent behaviors is valuable alongside optimizing one objective. This diversity leads to distinct high-performing systems, each emphasizing different tradeoffs among objectives.

Conclusion

We propose CEDAR, which casts complex system discovery as evolutionary search (Nisioti et al., 2024; Lehman et al., 2023) guided by MCTS: an LLM Editor acts as a variation operator and an LLM Judge as a fitness function. Empirically, on classical complex systems from social and biological modeling, CEDAR optimizes challenging multi-objective goals, outperforms black-box optimization without predefined formulas, and surfaces interpretable, step-by-step analyses of how the Editor and Judge reshape the system. Together, these capabilities open new directions for studying and exploring emergent phenomena in artificial life. Several limitations point to future work. The LLM Judge both scores candidates and shares a model class with the Editor, risking circularity, as the abstract-goal setting still relies on LLM judgment. We report trends rather than tight statistical claims, and the in-depth experiments focus on two systems. A natural next step is systematic benchmarking, along with classical evolutionary and multi-objective optimization methods, to delineate when LLM-based variation and evaluation are necessary rather than helpful.

References

- Akiba, T., Sano, S., Yanase, T., Ohta, T., and Koyama, M. (2019). Optuna: A Next-Generation Hyperparameter Optimization Framework. In *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*, pages 2623–2631.
- Anderson, V. and Johnson, L. (1997). *Systems Thinking Basics*. Pegasus Communications, Cambridge, MA.
- Arnold, R. D. and Wade, J. P. (2015). A Definition of Systems Thinking: A Systems Approach. *Procedia computer science*, 44:669–678.
- Barlas, Y. (1996). Formal Aspects of Model Validity and Validation in System Dynamics. *System Dynamics Review: The Journal of the System Dynamics Society*, 12(3):183–210.
- Bedau, M. A. (2003). Artificial Life: Organization, Adaptation and Complexity from the Bottom Up. *Trends in Cognitive Sciences*, 7(11):505–512.
- Bérard, C. (2010). Group Model Building Using System Dynamics: An Analysis of Methodological Frameworks. *Electronic Journal of Business Research Methods*, 8(1):35–45.
- Blackshaw, T. M., Davies, J. C., Spoerer, K. T., and Hirst, J. D. (2025). Enhancing Monte Carlo Tree Search for Retrosynthesis. *Journal of Chemical Information and Modeling*, 65(13):6537–6546.
- Brandfonbrener, D., Henniger, S., Raja, S., Prasad, T., Loughridge, C. R., Cassano, F., Hu, S. R., Yang, J., Byrd, W. E., Zinkov, R., and Amin, N. (2024). VerMCTS: Synthesizing Multi-Step Programs using a Verifier, a Large Language Model, and Tree Search. In *The 4th Workshop on Mathematical Reasoning and AI at NeurIPS'24*.
- Bubeck, S., Munos, R., Stoltz, G., and Szepesvári, C. (2011). X-Armed Bandits. *Journal of Machine Learning Research*, 12(46):1655–1695.
- Chaslot, G. M. J., Winands, M. H., Herik, H. J. v. d., Uiterwijk, J. W., and Bouzy, B. (2008). Progressive Strategies for Monte-Carlo Tree Search. *New Mathematics and Natural Computation*, 4(03):343–357.
- Chen, B., Li, C., Dai, H., and Song, L. (2020). Retro*: Learning Retrosynthetic Planning with Neural Guided A* Search. In III, H. D. and Singh, A., editors, *Proceedings of the 37th International Conference on Machine Learning*, volume 119 of *Proceedings of Machine Learning Research*, pages 1608–1616. PMLR.
- Chi, Y., Yang, K., and Klein, D. (2025). ThoughtSculpt: Reasoning with Intermediate Revision and Search. In Chiruzzo, L., Ritter, A., and Wang, L., editors, *Findings of the Association for Computational Linguistics: NAACL 2025*, pages 7700–7726, Albuquerque, New Mexico. Association for Computational Linguistics.
- Dainese, N., Merler, M., Alakuijala, M., and Marttinen, P. (2024). Generating Code World Models with Large Language Models Guided by Monte Carlo Tree Search. *Advances in Neural Information Processing Systems*, 37:60429–60474.
- Efroni, Y., Dalal, G., Scherrer, B., and Mannor, S. (2019). How to Combine Tree-Search Methods in Reinforcement Learning. *Proceedings of the AAAI Conference on Artificial Intelligence*, 33(01):3494–3501.
- Fawzi, A., Balog, M., Huang, A., Hubert, T., Romera-Paredes, B., Barekatin, M., Novikov, A., R. Ruiz, F. J., Schrittwieser, J., Swirszcz, G., et al. (2022). Discovering Faster Matrix Multiplication Algorithms with Reinforcement Learning. *Nature*, 610(7930):47–53.
- Fernando, C., Banarse, D. S., Michalewski, H., Osindero, S., and Rocktäschel, T. (2024). Promptbreeder: Self-Referential Self-Improvement via Prompt Evolution. In Salakhutdinov, R., Kolter, Z., Heller, K., Weller, A., Oliver, N., Scarlett, J., and Berkenkamp, F., editors, *Proceedings of the 41st International Conference on Machine Learning*, volume 235 of *Proceedings of Machine Learning Research*, pages 13481–13544. PMLR.
- Ford, D. N. and Sterman, J. D. (1998). Expert Knowledge Elicitation to Improve Formal and Mental Models. *System Dynamics Review: The Journal of the System Dynamics Society*, 14(4):309–340.
- Forrester, J. W. (1971). *World Dynamics*. Wright-Allen Press, Cambridge, Mass.
- Forrester, J. W. (2011). Mit system dynamics in education project. <https://www.merlot.org/merlot/viewMaterial.htm?id=518948>.
- Gan, B., Zhao, Y., Zhang, T., Huang, J., Yusu, L., Teo, S. X., Zhang, C., and Shi, W. (2025). MASTER: A Multi-agent System with LLM Specialized MCTS. In Chiruzzo, L., Ritter, A., and Wang, L., editors, *Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*, pages 9409–9426, Albuquerque, New Mexico. Association for Computational Linguistics.
- Gao, Z., Niu, B., He, X., Xu, H., Liu, H., Liu, A., Hu, X., and Wen, L. (2024). Interpretable Contrastive Monte Carlo Tree Search Reasoning. *arXiv preprint arXiv:2410.01707*.
- Gershenson, C. (2023). Emergence in Artificial Life. *Artificial Life*, 29(2):153–167.
- Güneralp, B. (2004). A Principle on Structure-Behavior Relations in System Dynamics Models. In *Proceedings of the 2004 International System Dynamics Conference, Oxford, UK*.
- Hannon, B. and Ruth, M. (2014). Modeling Dynamic Biological Systems. In *Modeling Dynamic Biological Systems*, pages 3–28. Springer.
- Hines, J. (1996). Molecules of Structure. *System Dynamics Group, Sloan School of Management, MIT*.
- Inoue, Y., Misaki, K., Imajuku, Y., Kuroki, S., Nakamura, T., and Akiba, T. (2025). Wider or Deeper? Scaling LLM Inference-Time Compute with Adaptive Branching Tree Search. In *The Thirty-ninth Annual Conference on Neural Information Processing Systems*.
- Katz, M., Kokel, H., Srinivas, K., and Sohrabi, S. (2024). Thought of Search: Planning with Language Models through the Lens of Efficiency. *Advances in Neural Information Processing Systems*, 37:138491–138568.

- Khattab, O., Singhvi, A., Maheshwari, P., Zhang, Z., Santhanam, K., A. S. V., Haq, S., Sharma, A., Joshi, T. T., Moazam, H., Miller, H., Zaharia, M., and Potts, C. (2024). DSPy: Compiling Declarative Language Model Calls into State-of-the-Art Pipelines. In *The Twelfth International Conference on Learning Representations*.
- Kocsis, L. and Szepesvári, C. (2006). Bandit Based Monte-Carlo Planning. In *European conference on machine learning*, pages 282–293. Springer.
- Kolson, K. (1996). The Politics of SimCity. *PS: Political Science & Politics*, 29(1):43–46.
- Kumar, A., Lu, C., Kirsch, L., Tang, Y., Stanley, K. O., Isola, P., and Ha, D. (2025). Automating the Search for Artificial Life with Foundation Models. *Artificial Life*, 31(3):368–396.
- Lai, Z. and Pu, Y. (2025). PriM: Principle-Inspired Material Discovery through Multi-Agent Collaboration. *arXiv preprint arXiv:2504.08810*.
- Langton, C. G. (2019). Artificial Life. In *Artificial Life*, pages 1–47. Routledge.
- Le, N., Erickson, P., Zhang, Y., Levin, M., and Bongard, J. (2025). Giving Simulated Cells a Voice: Evolving Prompt-to-Intervention Models for Cellular Control. In *Proceedings of the Genetic and Evolutionary Computation Conference Companion, GECCO '25 Companion*, page 2327–2335, New York, NY, USA. Association for Computing Machinery.
- Le, N. H., Erickson, P., Zhang, Y., Levin, M., and Bongard, J. (2026). ZapGPT: Free-form Language Prompting for Simulated Cellular Control. In *Proceedings of the 2025 8th International Conference on Computational Intelligence and Intelligent Systems, CHIS '25*, page 119–126, New York, NY, USA. Association for Computing Machinery.
- Lehman, J., Gordon, J., Jain, S., Ndousse, K., Yeh, C., and Stanley, K. O. (2023). Evolution through Large Models. In *Handbook of Evolutionary Machine Learning*, pages 331–366. Springer.
- Li, G., Hammoud, H., Itani, H., Khizbullin, D., and Ghanem, B. (2023). CAMEL: Communicative Agents for "Mind" Exploration of Large Language Model Society. In Oh, A., Naumann, T., Globerson, A., Saenko, K., Hardt, M., and Levine, S., editors, *Advances in Neural Information Processing Systems*, volume 36, pages 51991–52008. Curran Associates, Inc.
- Li, J., Le, H., Zhou, Y., Xiong, C., Savarese, S., and Sahoo, D. (2025a). CodeTree: Agent-guided Tree Search for Code Generation with Large Language Models. In Chiruzzo, L., Ritter, A., and Wang, L., editors, *Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*, pages 3711–3726, Albuquerque, New Mexico. Association for Computational Linguistics.
- Li, Q., Xia, W., Dai, X., Du, K., Liu, W., Wang, Y., Tang, R., Yu, Y., and Zhang, W. (2025b). RethinkMCTS: Refining Erroneous Thoughts in Monte Carlo Tree Search for Code Generation. In Christodoulopoulos, C., Chakraborty, T., Rose, C., and Peng, V., editors, *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, pages 8092–8110, Suzhou, China. Association for Computational Linguistics.
- Liang, Z., Wei, F., Xu, W., Chen, L., Qian, Y., and Wu, X. (2025). I-MCTS: Enhancing Agentic AutoML via Introspective Monte Carlo Tree Search. *arXiv preprint arXiv:2502.14693*.
- Lisy, V., Kovarik, V., Lanctot, M., and Bosansky, B. (2013). Convergence of Monte Carlo Tree Search in Simultaneous Move Games. In Burges, C., Bottou, L., Welling, M., Ghahramani, Z., and Weinberger, K., editors, *Advances in Neural Information Processing Systems*, volume 26. Curran Associates, Inc.
- Liu, N.-Y. G. and Keith, D. R. (2025). Leveraging Large Language Models for Automated Causal Loop Diagram Generation: Enhancing System Dynamics Modeling through Curated Prompting Techniques. *arXiv preprint arXiv:2503.21798*.
- Liu, T. J., Boulle, N., Sarfati, R., and Earls, C. (2024). LLMs Learn Governing Principles of Dynamical Systems, Revealing an In-Context Neural Scaling Law. In Al-Onaizan, Y., Bansal, M., and Chen, Y.-N., editors, *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, pages 15097–15117, Miami, Florida, USA. Association for Computational Linguistics.
- Luo, X., Chen, B., Wang, H., Xiao, Z., Zhang, M., and Sun, Y. (2025). How Do Large Language Models Perform in Dynamical System Modeling. In Chiruzzo, L., Ritter, A., and Wang, L., editors, *Findings of the Association for Computational Linguistics: NAACL 2025*, pages 866–880, Albuquerque, New Mexico. Association for Computational Linguistics.
- Mankowitz, D. J., Michi, A., Zhernov, A., Gelmi, M., Selvi, M., Paduraru, C., Leurent, E., Iqbal, S., Lespiau, J.-B., Ahern, A., et al. (2023). Faster Sorting Algorithms Discovered Using Deep Reinforcement Learning. *Nature*, 618(7964):257–263.
- McLachlan, N. W. (2014). *Laplace transforms and their applications to differential equations*. Courier Corporation.
- Meert, W., Hendrickx, K., Van Craenendonck, T., Robberechts, P., Blockeel, H., and Davis, J. (2022). DTAIDistance (Version v2.3.5).
- MIT System Dynamics in Education Project (1998). Road Maps: A Guide to Learning System Dynamics. <http://www.clexchange.org/curriculum/roadmaps/>. System Dynamics Group, Sloan School of Management, Massachusetts Institute of Technology. Accessed: 2025-08-30.
- Nasir, M. U., Earle, S., Togelius, J., James, S., and Cleghorn, C. (2024). LLMatic: Neural Architecture Search Via Large Language Models And Quality Diversity Optimization. In *Proceedings of the Genetic and Evolutionary Computation Conference, GECCO '24*, pages 1110–1118, New York, NY, USA. Association for Computing Machinery.
- Nisioti, E., Glanois, C., Najarro, E., Dai, A., Meyerson, E., Pedersen, J. W., Teodorescu, L., Hayes, C. F., Sudhakaran, S., and Risi, S. (2024). From Text to Life: On the Reciprocal Relationship between Artificial Life and Large Language Models. In *Artificial Life Conference Proceedings*, volume 36, page 39. MIT Press.
- Ogata, K. (2004). *System Dynamics*. Pearson Education, 4 edition.
- Peterson, S. (2003). Barry Richmond, System Dynamics and Public Policy. In *21st System Dynamics Conference*, pages 1–14.

- Radzicki, M. J. and Taylor, R. A. (2008). Origin of System Dynamics: Jay W. Forrester and the History of System Dynamics. *US Department of Energy's introduction to system dynamics*.
- Ratanamahatana, C. A. and Keogh, E. (2004). Everything You Know About Dynamic Time Warping Is Wrong. In *Third workshop on mining temporal and sequential data*, volume 32, pages 1–11. Citeseer Seattle.
- Richmond, B. (1985). STELLA: Software for Bringing System Dynamics to the Other 98%. In *Proceedings of the 1985 International Conference of the System Dynamics Society*, pages 706–718. System Dynamics Society 49 Bedford Road, Lincoln, MA 01773 USA.
- Ruth, M. and Hannon, B. (2012). *Modeling Dynamic Economic Systems*. Springer.
- Sakoe, H. and Chiba, S. (1978). Dynamic Programming Algorithm Optimization for Spoken Word Recognition. *IEEE Transactions on Acoustics, Speech, and Signal Processing*, 26(1):43–49.
- Schoenberg, W., Davidsen, P., and Eberlein, R. (2020). Understanding Model Behavior Using the Loops That Matter Method. *System Dynamics Review*, 36(2):158–190.
- Schrittwieser, J., Antonoglou, I., Hubert, T., Simonyan, K., Sifre, L., Schmitt, S., Guez, A., Lockhart, E., Hassabis, D., Graepel, T., et al. (2020). Mastering Atari, Go, Chess and Shogi by Planning with a Learned Model. *Nature*, 588(7839):604–609.
- Schünemann, C., Johanning, S., Reger, E., Herold, H., and Bruckner, T. (2024). Complex System Policy Modelling Approaches for Policy Advice—Comparing Systems Thinking, System Dynamics and Agent-based Modelling. *Political Research Exchange*, 6(1):2387438.
- Shinn, N., Cassano, F., Gopinath, A., Narasimhan, K., and Yao, S. (2023). Reflexion: Language Agents with Verbal Reinforcement Learning. In Oh, A., Naumann, T., Globerson, A., Saenko, K., Hardt, M., and Levine, S., editors, *Advances in Neural Information Processing Systems*, volume 36, pages 8634–8652. Curran Associates, Inc.
- Shojaee, P., Nguyen, N.-H., Meidani, K., Farimani, A. B., Doan, K. D., and Reddy, C. K. (2025). LLM-SRBench: A New Benchmark for Scientific Equation Discovery with Large Language Models. In *Forty-second International Conference on Machine Learning*.
- Silver, D., Huang, A., Maddison, C. J., Guez, A., Sifre, L., Van Den Driessche, G., Schrittwieser, J., Antonoglou, I., Panneershelvam, V., Lanctot, M., et al. (2016). Mastering the Game of Go with Deep Neural Networks and Tree Search. *nature*, 529(7587):484–489.
- Silver, D., Hubert, T., Schrittwieser, J., Antonoglou, I., Lai, M., Guez, A., Lanctot, M., Sifre, L., Kumaran, D., Graepel, T., Lillicrap, T., Simonyan, K., and Hassabis, D. (2017a). Mastering Chess and Shogi by Self-Play with a General Reinforcement Learning Algorithm. *arXiv preprint arXiv:1712.01815*.
- Silver, D., Schrittwieser, J., Simonyan, K., Antonoglou, I., Huang, A., Guez, A., Hubert, T., Baker, L., Lai, M., Bolton, A., et al. (2017b). Mastering the Game of Go without Human Knowledge. *nature*, 550(7676):354–359.
- Song, X., Zhang, S., and Yu, T. (2025). ReKG-MCTS: Reinforcing LLM Reasoning on Knowledge Graphs via Training-Free Monte Carlo Tree Search. In Che, W., Nabende, J., Shutova, E., and Pilehvar, M. T., editors, *Findings of the Association for Computational Linguistics: ACL 2025*, pages 9288–9306, Vienna, Austria. Association for Computational Linguistics.
- Srinivasan, K. K., Le, N., Bongard, J., and Blackiston, D. (2025). Evolving agents to generate and falsify hypotheses of biological self-assembly. In *Artificial Life Conference Proceedings*, volume 37, page 24. MIT Press.
- Sterman, J. D. (2000). *Business Dynamics: Systems Thinking and Modeling for a Complex World*. McGraw-Hill Education.
- Tong, Z., Ding, Z., and Wei, W. (2025). EvoPrompt: Evolving Prompts for Enhanced Zero-Shot Named Entity Recognition with Large Language Models. In Rambow, O., Wanner, L., Apidianaki, M., Al-Khalifa, H., Eugenio, B. D., and Schockaert, S., editors, *Proceedings of the 31st International Conference on Computational Linguistics*, pages 5136–5153, Abu Dhabi, UAE. Association for Computational Linguistics.
- Walter, W. (2013). *Ordinary differential equations*, volume 182. Springer Science & Business Media.
- Wu, F., Xuan, W., Qi, H., Lu, X., Tu, A., Li, L. E., and Choi, Y. (2025). DeepSearch: Overcome the Bottleneck of Reinforcement Learning with Verifiable Rewards via Monte Carlo Tree Search. *arXiv preprint arXiv:2509.25454*.
- Xu, B., Lin, Y., Li, Y., and Gao, Y. (2025). SRA-MCTS: Self-driven Reasoning Augmentation with Monte Carlo Tree Search for Code Generation. In Kwok, J., editor, *Proceedings of the Thirty-Fourth International Joint Conference on Artificial Intelligence, IJCAI-25*, pages 8678–8686. International Joint Conferences on Artificial Intelligence Organization. Main Track.
- Xu, Y., Liu, Y., and Sun, H. (2024). Reinforcement Symbolic Regression Machine. In *The Twelfth International Conference on Learning Representations*.
- Yamada, Y., Lange, R. T., Lu, C., Hu, S., Lu, C., Foerster, J., Clune, J., and Ha, D. (2025). The AI Scientist-v2: Workshop-Level Automated Scientific Discovery via Agentic Tree Search. *arXiv preprint arXiv:2504.08066*.
- Zagonel, A. A. (2002). Model Conceptualization in Group Model Building: A Review of the Literature Exploring the Tension Between Representing Reality and Negotiating a Social Order. In *Proceedings of the 20th International System Dynamics Conference*, volume 51, pages 170–182.
- Zelikman, E., Wu, Y., Mu, J., and Goodman, N. (2022). STaR: Bootstrapping Reasoning With Reasoning. In Koyejo, S., Mohamed, S., Agarwal, A., Belgrave, D., Cho, K., and Oh, A., editors, *Advances in Neural Information Processing Systems*, volume 35, pages 15476–15488. Curran Associates, Inc.
- Zhao, Z., Lee, W. S., and Hsu, D. (2023). Large Language Models as Commonsense Knowledge for Large-Scale Task Planning. *Advances in neural information processing systems*, 36:31967–31987.
- Zheng, Z., Xie, Z., Wang, Z., and Hooi, B. (2025). Monte Carlo Tree Search for Comprehensive Exploration in LLM-Based Automatic Heuristic Design. *arXiv preprint arXiv:2501.08603*.

- Zhou, A., Yan, K., Shlapentokh-Rothman, M., Wang, H., and Wang, Y.-X. (2024). Language Agent Tree Search Unifies Reasoning, Acting, and Planning in Language Models. In *Forty-first International Conference on Machine Learning*.
- Zhou, Y., Muresanu, A. I., Han, Z., Paster, K., Pitis, S., Chan, H., and Ba, J. (2023). Large Language Models are Human-Level Prompt Engineers. In *The Eleventh International Conference on Learning Representations*.
- Zhu, X., Shi, Y., and Zhong, Y. (2025). An EKF Prediction of COVID-19 Propagation under Vaccinations and Viral Variants. *Mathematics and Computers in Simulation*, 231:221–238.

APPENDIX

Appendix Overview

The appendix is organized as follows. Since appendix references are not included in the main submission text, this overview serves as a guide to locating supplementary material.

Python Implementation of Complex Systems (Section). Details the unified Python-based representation, including a minimal code example, the full *World Dynamics* system code, and the documentation and domain-specific math primitives provided to LLMs.

Method Details (Section). Contains the full prompt templates for LLM Judge and LLM Editor (Section), and the original node selection hyperparameter rationale (Section).

Theoretical Connection: Full Details (Section). Presents the full derivations of node selection as a UCT generalization, LLM Editor as a learned transition kernel, and LLM Judge as a learned value function, with equations and discussion of open theoretical problems.

Experiment Details (Section). Contains dataset variable statistics (Section), the full expansion strategy set (Section), engineering decisions for scalability and stability (Section), and computational cost and runtime (Section).

Fitting an Abstract Goal: Details (Section). Contains the *World Dynamics* system structure and variable definitions (Section), the exact natural-language goal text (Section), and a quantitative demonstration of competing-subgoal challenges (Section).

Fitting a Concrete Record: Details (Section). Contains the ground truth system implementation (Section), the three levels of Optuna baseline code (Section), the DTW metric definition (Section), stochasticity analysis across seeds (Section), and an additional Optuna run (Section).

Interpretability Analysis (Section). Contains full LLM Judge and Editor transcripts from the *World Dynamics* optimization run.

Python Implementation of Complex Systems

Here we detail our proposed unified representation of complex systems based on a restricted subset of Python programs. Concretely, the components are divided into four parts: (1) initialization, a loop of (2) intermediate computations, (3) derivative evaluations, and (4) Euler-method integration. We show a minimal example of a program with these components below:

```
1 # (1) initialization
2 dt = 0.01
3 t0 = 0.0
4 tf = 120.0
5 K = 100.0
6 R = 0.1
7 N = 10.0
8
9 t = t0
10 while t <= tf + 1e-12:
11
12     # (2) intermediate computations
13     carrying_capacity_factor = 1.0 - N / K
14     growth_rate = R * N * carrying_capacity_factor
15
16     # (3) derivative evaluations
17     dN = growth_rate # dN = R * N * (1 - N/K)
18
19     # (4) Euler integration
20     N = N + dt * dN
21     t += dt
```

In practice, we find that including extensive comments in the program is not only helpful for humans to understand, but also beneficial for LLMs. Furthermore, to help LLMs' tool use, we extend the code with further documentation and predefined math primitives, such as common math functions, delay, and smoothing functions that are frequently used in complex systems. The benefits are twofold: (1) Such comments and documents help LLMs to understand the complex system's semantics for better optimization, and the standardized functions ease LLMs' job in generating code for new systems, and (2) more importantly, it instructs LLMs on what restrictions should be enforced, which is important since several critical restrictions, such as preserving the Euler-method integration, must be respected for the program to execute successfully.

We show our representation of the complex system of the *World Dynamics* system (Forrester, 1971). As it is the one with the largest number of variables in our dataset, this code below demonstrates the full-fledged form of our representation, including how we enforce the constraints for LLMs and providing semantics to help the optimization. We show the code in two parts: First, we show the code that directly describes the complex system's dynamics:


```

1 # ===== BEGIN CONFIG =====
2 # Simulation configuration parameters (DO NOT CHANGE - these are set by the system)
3 dt = 0.2 # time step size
4 t0 = 1900.0 # start time
5 tf = 2100.0 # end time
6 seed = 1234 # random seed (set None for nondeterministic runs)
7 # ===== END CONFIG =====
8
9 # ===== BEGIN STATE =====
10 # STATE VARIABLES - integrated over time (must define derivatives below)
11 P = 1.65e9 # population (people) # NECESSARY
12 NR = 900e9 # natural resources (natural resource units) # NECESSARY
13 CI = 0.4e9 # capital investment (capital units) # NECESSARY
14 POL = 0.2e9 # pollution (pollution units) # NECESSARY
15 CIAF = 0.2 # capital-investment-in-agriculture fraction (dimensionless) # NECESSARY
16 # ===== END STATE =====
17
18 # ===== BEGIN ALGEBRAIC (OPTIONAL) =====
19 # Variables are now declared in the HELPERS section within the simulation loop
20
21 # ===== BEGIN PARAMS =====
22 # CONSTANT PARAMETERS used in equations
23 # (All constants are inlined in the code)
24 # ===== END PARAMS =====
25
26 # ===== MAIN LOOP =====
27 t = t0
28 while t <= tf + 1e-12:
29     # ===== BEGIN INPUTS_T =====
30     # TIME-DEPENDENT INPUTS - use wrappers like sin(), exp(), graph()
31     # (No time-dependent inputs in this model)
32     # ===== END INPUTS_T =====
33
34     # ===== BEGIN INPUTS_RND =====
35     # RANDOM INPUTS - use gauss(), uniform() wrappers (no STATE usage).
36     # (No random inputs in this model)
37     # ===== END INPUTS_RND =====
38
39     # ===== BEGIN HELPERS =====
40     # HELPERS - All computed variables (algebraic and intermediate expressions)
41     # in dependency order.
42
43     # Basic ratios and derived quantities
44     CIR = CI / P # capital-investment ratio (capital units/person)
45     CR = P / (135e6 * 26.5) # crowding ratio (dimensionless)
46     NRFR = NR / 900e9 # natural-resource fraction remaining (dimensionless)
47     POLR = POL / 3.6e9 # pollution ratio (dimensionless)
48
49     # Material standard of living components
50     CIAF_current = CIAF # current capital-investment-in-agriculture fraction
51     NREM = graph(
52         NRFR, ((0, 0), (0.25, 0.15), (0.5, 0.5), (0.75, 0.85), (1, 1))
53     ) # natural-resource-extraction multiplier
54     ECIR = CIR * (1 - CIAF_current) * NREM / (1 - 0.3) # effective-capital-investment ratio
55     MSL = ECIR / 1 # material standard of living (dimensionless)
56
57     # Birth rate multipliers
58     BRMM = graph(
59         MSL, ((0, 1.2), (1, 1), (2, 0.85), (3, 0.75), (4, 0.7), (5, 0.7))
60     ) # birth-rate-from-material multiplier
61     BRMC = graph(
62         CR, ((0, 1.05), (1, 1), (2, 0.9), (3, 0.7), (4, 0.6), (5, 0.55))
63     ) # birth-rate-from-crowding multiplier
64     BRPM = graph(
65         POLR, ((0, 1.02), (10, 0.9), (20, 0.7), (30, 0.4), (40, 0.25), (50, 0.15), (60, 0.1))
66     ) # birth-rate-from-pollution multiplier
67
68     # Food ratio components
69     CIRA = CIR * CIAF_current / 0.3 # capital-investment ratio in agriculture
70     FPCI = graph(
71         CIRA, ((0, 0.5), (1, 1), (2, 1.4), (3, 1.7), (4, 1.9), (5, 2.05), (6, 2.2))
72     ) # food potential from capital investment
73     FCM = graph(CR, ((0, 2.4), (1, 1), (2, 0.6), (3, 0.4), (4, 0.3), (5, 0.2))) # food-from-crowding multiplier
74     FPM = graph(
75         POLR, ((0, 1.02), (10, 0.9), (20, 0.65), (30, 0.35), (40, 0.2), (50, 0.1), (60, 0.05))
76     ) # food-from-pollution multiplier
77     FR = FPCI * FCM * FPM * (1 if t >= 1970 else 1) / 1 # food ratio
78     BRFM = graph(FR, ((0, 0), (1, 1), (2, 1.6), (3, 1.9), (4, 2))) # birth-rate-from-food multiplier
79
80     # Death rate multipliers

```

```

81 DRMM = graph(
82     MSL,
83     (
84         (0, 3),
85         (0.5, 1.8),
86         (1, 1),
87         (1.5, 0.8),
88         (2, 0.7),
89         (2.5, 0.6),
90         (3, 0.53),
91         (3.5, 0.5),
92         (4, 0.5),
93         (4.5, 0.5),
94         (5, 0.5),
95     ),
96 ) # death-rate-from-material multiplier
97 DRCM = graph(CR, ((0, 0.9), (1, 1), (2, 1.2), (3, 1.5), (4, 1.9), (5, 3))) # death-rate-from-crowding multiplier
98 DRPM = graph(
99     POLR, ((0, 0.92), (10, 1.3), (20, 2), (30, 3.2), (40, 4.8), (50, 6.8), (60, 9.2))
100 ) # death-rate-from-pollution multiplier
101 DRFM = graph(
102     FR, ((0, 30), (0.25, 3), (0.5, 2), (0.75, 1.4), (1, 1), (1.25, 0.7), (1.5, 0.6), (1.75, 0.5), (2, 0.5))
103 ) # death-rate-from-food multiplier
104
105 # Natural resource usage multiplier
106 NRMM = graph(
107     MSL, ((0, 0), (1, 1), (2, 1.8), (3, 2.4), (4, 2.9), (5, 3.3), (6, 3.6), (7, 3.8), (8, 3.9), (9, 3.95), (10,
108         4))
109 ) # natural-resource-from-material multiplier
110
111 # Capital investment multiplier
112 CIM = graph(MSL, ((0, 0.1), (1, 1.0), (2, 1.8), (3, 2.4), (4, 2.8), (5, 3))) # capital-investment multiplier
113
114 # Pollution components
115 POLCM = graph(CIR, ((0, 0.05), (1, 1), (2, 3), (3, 5.4), (4, 7.4), (5, 8))) # pollution-from-capital multiplier
116 POLAT = graph(
117     POLR, ((0, 0.6), (10, 2.5), (20, 5), (30, 8), (40, 11.5), (50, 15.5), (60, 20))
118 ) # pollution-absorption time
119
120 # Quality of life components
121 QLM = graph(MSL, ((0, 0.2), (1, 1), (2, 1.7), (3, 2.3), (4, 2.7), (5, 2.9))) # quality of life from material
122 QLC = graph(
123     CR,
124     (
125         (0, 2),
126         (0.5, 1.3),
127         (1, 1),
128         (1.5, 0.75),
129         (2, 0.55),
130         (2.5, 0.45),
131         (3, 0.38),
132         (3.5, 0.3),
133         (4, 0.25),
134         (4.5, 0.22),
135         (5, 0.2),
136     ),
137 ) # quality of life from crowding
138 QLF = graph(FR, ((0, 0), (1, 1), (2, 1.8), (3, 2.4), (4, 2.7))) # quality of life from food
139 QLP = graph(
140     POLR, ((0, 1.04), (10, 0.85), (20, 0.6), (30, 0.3), (40, 0.15), (50, 0.05), (60, 0.02))
141 ) # quality of life from pollution
142 QL = 1 * QLM * QLC * QLF * QLP # quality of life
143
144 # Capital investment fraction adjustment components
145 CFIFR = graph(FR, ((0, 1), (0.5, 0.6), (1, 0.3), (1.5, 0.15), (2, 0.1))) # capital fraction indicated by food
146 ratio
147 CIQR = graph(QLM / QLF, ((0, 0.7), (0.5, 0.8), (1, 1), (1.5, 1.5), (2, 2))) # capital-investment-from-quality
148 ratio
149
150 # ===== END HELPERS =====
151
152 # ===== BEGIN DERIVATIVES =====
153 # DERIVATIVES for each STATE variable.
154
155 # Birth and death rates
156 BR = P * (0.04 if t >= 1970 else 0.04) * BRFM * BRMM * BRCM * BRPM # birth rate
157 DR = P * (0.028 if t >= 1970 else 0.028) * DRMM * DRPM * DRFM * DRCM # death rate
158
159 # Natural resource usage rate
160 NRUR = P * (1 if t >= 1970 else 1) * NRMM # natural-resource-usage rate

```

```

159 # Capital investment flows
160 CIG = P * CIM * (0.05 if t >= 1970 else 0.05) # capital-investment generation
161 CID = CI * (0.025 if t >= 1970 else 0.025) # capital-investment discard
162
163 # Pollution flows
164 POLG = P * (1 if t >= 1970 else 1) * POLCM # pollution generation
165 POLA = POL / POLAT # pollution absorption
166
167 # State derivatives
168 dP = BR - DR
169 dNR = -NRUR
170 dCI = CIG - CID
171 dPOL = POLG - POLA
172 dCIAF = (1 / 15) * (CFIFR * CIQR - CIAF)
173 # ===== END DERIVATIVES =====
174
175 # ----- Euler integration (engine; do not edit) -----
176 P = P + dt * dP
177 NR = NR + dt * dNR
178 CI = CI + dt * dCI
179 POL = POL + dt * dPOL
180 CIAF = CIAF + dt * dCIAF
181
182 # Advance time
183 t += dt

```

Then, we show the documentation and math functions. The code above is placed between `BEGIN WHOLE SYSTEM` and `END WHOLE SYSTEM`.

```

1 # =====
2 # sim_base.py - Minimal Euler simulator with edit markers
3 #
4 # PURPOSE:
5 # - Self-contained forward Euler simulator:
6 #     X(t+dt) = X(t) + dt * dX/dt
7 #
8 # HOW TO EDIT:
9 # - Only change code between the "BEGIN ... / END ..." markers below.
10 # - Do NOT modify imports, wrappers, loop structure, Euler integration, or OUTPUT section.
11 # - Keep STATE, ALGEBRAIC, HELPERS, and DERIVATIVES consistent.
12 # - NEVER change simulation configuration parameters (dt, t0, tf, seed) in the CONFIG section.
13 #
14 # VARIABLE MODIFICATION RULES:
15 # - STATE VARIABLES: You can add new state variables. You can delete state variables ONLY if they
16 #   are not marked as "# NECESSARY" in comments. All state variables must have derivatives.
17 # - ALGEBRAIC VARIABLES: You can add new algebraic variables. You can delete algebraic variables
18 #   ONLY if they are not marked as "# NECESSARY" in comments.
19 # - HELPERS: You can freely add, remove, and modify helper variables. You can change their
20 #   computation formulas and reorder them, as long as the code remains valid.
21 # - When converting from other systems: Mark essential state and algebraic variables with
22 #   "# NECESSARY" comments so optimization/editing knows which variables cannot be deleted.
23 #
24 # VARIABLE TYPES:
25 # - STATE: variables integrated over time (must have derivatives).
26 # - INPUTS_T: pure functions of time (cannot depend on STATE).
27 # - INPUTS_RND: fresh random values each step (cannot depend on STATE).
28 # - HELPERS: all computed variables (algebraic and intermediate expressions)
29 #   in dependency order. Can use wrappers like sin(), exp(), graph(),
30 #   delay(), smth1() - e.g. nonlinear damping from a table.
31 # - DERIVATIVES: d/dt for each STATE variable.
32 #
33 # TRACE CONTENT (per step, in fixed order):
34 #   1) t           - current time
35 #   2) STATE variables
36 #   3) HELPERS (all computed variables)
37 #   4) DERIVATIVES
38 #
39 # OUTPUT:
40 # - If --csv <path> is provided -> write full simulation to that CSV file.
41 # - Otherwise -> print the raw CSV (header + all rows) to stdout.
42 # - Columns/ordering come from the snapshot dict defined inside the loop.
43 # =====
44
45 import math
46 import random
47 import argparse
48 import pandas as pd # used only for CSV export
49 from typing import Iterable, Tuple
50
51 # ----- Wrapper Functions -----

```

```

52 # These wrappers hide Python semantics; LLMs should ONLY call these.
53
54 def sin(x: float) -> float:
55     """Sine function (angle in radians)."""
56     return math.sin(x)
57
58 def cos(x: float) -> float:
59     """Cosine function (angle in radians)."""
60     return math.cos(x)
61
62 def exp(x: float) -> float:
63     """Exponential function e^x."""
64     return math.exp(x)
65
66 def tanh(x: float) -> float:
67     """Hyperbolic tangent."""
68     return math.tanh(x)
69
70 def sqrt(x: float) -> float:
71     """Square root."""
72     return math.sqrt(x)
73
74 def log(x: float) -> float:
75     """Natural logarithm."""
76     return math.log(x)
77
78 def gauss(std: float) -> float:
79     """Gaussian random variable with mean=0 and standard deviation=std."""
80     return random.gauss(0.0, std)
81
82 def uniform(lo: float, hi: float) -> float:
83     """Uniform random variable between lo and hi."""
84     return random.uniform(lo, hi)
85
86 def graph(v: float, table: Iterable[Tuple[float, float]]) -> float:
87     """
88     Lookup helper with linear interpolation.
89     - table is an iterable of (x, y) points sorted by x.
90     - If v < x0, return y0.
91     - If v > xN, return yN.
92     - Else, linearly interpolate between nearest points.
93     """
94     table = list(table)
95     if not table:
96         raise ValueError("graph() called with empty table")
97
98     if v <= table[0][0]:
99         return table[0][1]
100     if v >= table[-1][0]:
101         return table[-1][1]
102
103     for i in range(len(table) - 1):
104         x0, y0 = table[i]
105         x1, y1 = table[i + 1]
106         if x0 <= v <= x1:
107             if x1 == x0:
108                 return y0
109             frac = (v - x0) / (x1 - x0)
110             return y0 + frac * (y1 - y0)
111     return table[-1][1]
112
113 # ----- Delay/Smooth System -----
114 # STELLA-compatible delay/smooth functions with time-based semantics.
115 # - All functions require a 'name' parameter for unique identification
116 # - Updates occur when functions are called during helper computation
117 # - Fixed delays use time-indexed history with linear interpolation
118 # - Smooth functions use stock-based integration with proper dt scaling
119 # - All functions are dt-independent (same behavior regardless of step size)
120
121 _delay_states = {} # Registry for all delay/smooth function states
122
123 def delay(input_val: float, delay_time: float, name: str, initial_value: float = None) -> float:
124     """Fixed lag delay - returns input value from delay_time ago."""
125     key = f"delay_{name}_{delay_time}"
126
127     if key not in _delay_states:
128         init_val = input_val if initial_value is None else initial_value
129         _delay_states[key] = {
130             'history': [(t, init_val)],
131             'last_t': t
132         }

```

```

133 state = _delay_states[key]
134
135 # Add current input to history (only if time advanced)
136 if t > state['last_t']:
137     state['history'].append((t, input_val))
138     state['last_t'] = t
139
140 # Clean old history beyond delay time
141 cutoff_time = t - delay_time
142 state['history'] = [(t_hist, val) for t_hist, val in state['history']
143                     if t_hist >= cutoff_time]
144
145 # Find value at t - delay_time using linear interpolation
146 target_time = t - delay_time
147 if not state['history'] or target_time <= state['history'][0][0]:
148     return state['history'][0][1]
149
150 for i in range(len(state['history']) - 1):
151     t1, v1 = state['history'][i]
152     t2, v2 = state['history'][i + 1]
153     if t1 <= target_time <= t2:
154         if t2 == t1:
155             return v1
156         frac = (target_time - t1) / (t2 - t1)
157         return v1 + frac * (v2 - v1)
158
159 return state['history'][-1][1]
160
161 def smth1(input_val: float, smooth_time: float, name: str, initial_value: float = None) -> float:
162     """First-order exponential smooth - stock-based smoothing process."""
163     key = f"smth1_{name}_{smooth_time}"
164
165     if key not in _delay_states:
166         init_val = input_val if initial_value is None else initial_value
167         _delay_states[key] = {
168             'smooth_of_input': init_val, # The stock being smoothed
169             'last_t': t
170         }
171
172     state = _delay_states[key]
173
174     if t > state['last_t']:
175         dt_step = t - state['last_t']
176
177         # SMTH1 equations from STELLA:
178         # Change_in_Smooth = (Input - Smooth_of_Input) / Averaging_Time
179         change_in_smooth = (input_val - state['smooth_of_input']) / smooth_time if smooth_time > 0 else 0.0
180
181         # Stock integration: Smooth_of_Input = Smooth_of_Input + dt * Change_In_Smooth
182         state['smooth_of_input'] += dt_step * change_in_smooth
183         state['last_t'] = t
184
185     # SMTH1 returns the smoothed stock value
186     return state['smooth_of_input']
187
188
189 # ===== BEGIN WHOLE SYSTEM =====
190 #
191 #
192 #
193 # ===== END WHOLE SYSTEM =====
194
195 # ===== OUTPUT (FIXED) =====
196 # DO NOT EDIT THIS SECTION.
197 def _parse_args():
198     ap = argparse.ArgumentParser(description="Run Euler simulation and export CSV.")
199     ap.add_argument("--csv", dest="csv_path", default=None,
200                     help="Write results to this CSV path. If omitted, prints CSV to stdout.")
201     return ap.parse_args()
202
203 if __name__ == "__main__":
204     args = _parse_args()
205     df = pd.DataFrame(trace) # column order = insertion order of dict
206     if args.csv_path:
207         df.to_csv(args.csv_path, index=False, float_format='%.6f')
208         print(f"Simulation complete. Results written to {args.csv_path}")
209     else:
210         print(df.to_csv(index=False, float_format='%.6f'), end="")

```

Method Details: LLM Contextualization, Scalability and Stability

LLM Contextualization

In our design, we pair LLM Judge, which performs evaluations of one complex system, with LLM Editor, which produces new modified systems to better achieve the goal. Besides the enriched code-based representation detailed in Appendix , we also need well-crafted prompts to help LLMs to understand the semantics of complex systems and to generate meaningful modifications towards the goal. In fact, since the code is in the context of the prompt, both the prompt design and code representation can be seen as a coherent *contextualization* of the information. Also, we find that this contextualization should include the tree context, as our preliminary studies found that without tree context, the expansion process fails to produce meaningful improvements.

In conclusion, the prompt templates for LLM Judge and LLM Editor are listed below respectively:

Prompt for LLMJUDGE

ANALYZE SIMULATION RESULTS FOR TREE-BASED OPTIMIZATION

GOAL:

{goal}

CURRENT CODE:

{code}

SIMULATION RESULTS:

{csv_data}

TREE CONTEXT:

{tree_context}

=== For your reference ===

Format for code in tree context:

The tree context shows only the differences in each reference node compared to current code.

Since the full current code is shown above, diffs only display lines that are different in reference nodes:

- Lines starting with "+" show what the reference node has instead of current code
- "Code identical to current node" means no differences exist

When analyzing this node's performance, consider:

- How well this specific code variant achieves the optimization goal
- Performance relative to ALL other nodes in the reference (for calibrated scoring)
- Whether this represents meaningful progress in the search space

SCORING GUIDELINES:

ABSOLUTE PERFORMANCE SCORING (depth-constrained scale)

Primary Principle: Score based on how well this code achieves the optimization goal, regardless of other nodes in the tree.

SCORING SCALE (use 2 decimal places like 7.25, 12.75):

- 0.00-2.00: Failed progress - no meaningful progress toward the goal
- 2.00-4.00: Partial progress - some improvement but far from achieving the goal
- 4.00-6.00: Moderate progress - measurable improvement and moving toward the goal
- 6.00-8.00: Good progress - clear advancement with substantial goal achievement
- 8.00-10.00: Excellent progress - meets most requirements of the optimization goal
- 10.00+: VERY GOOD performance - exceeds the goal expectations significantly
- 20.00+: EXCEPTIONAL performance - far surpasses what the goal was asking for

IMPORTANT CONSTRAINT: Maximum possible score is $\text{score} \leq 10.0 + 2.5 * \text{depth}$. Within this limit, high scores are encouraged when performance truly merits them.

Use the tree context information to ensure your scoring is well-calibrated relative

to all explored alternatives.

=== Task ===

Please provide your analysis in this format:

REASONING: [Start with a brief summary of key findings in the first 200 characters, then provide detailed reasoning about how well this code variant achieves the optimization goal]

SCORE: [numerical score based on the scoring guidelines above]

Prompt for LLMEDITOR

SUGGEST CODE MODIFICATIONS FOR TREE SEARCH OPTIMIZATION

GOAL:

{goal}

CURRENT CODE:

{code}

SIMULATION RESULTS:

{csv_data}

TREE CONTEXT:

{tree_context}

=== For your reference ===

Format for code in tree context:

The tree context shows only the differences in each reference node compared to current code.

Since the full current code is shown above, diffs only display lines that are different in reference nodes:

- Lines starting with "+" show what the reference node has instead of current code
- "Code identical to current node" means no differences exist

How to modify the code to better achieve the goal within the tree search context:

Focus on:

- Parameters within the marked BEGIN/END edit sections
- State variables, constants, and simulation configuration
- Derivative equations and helper expressions
- Table values and time-dependent inputs
- Learning from tree context and reference node outcomes

When you modify the code:

- Faithfully follow the CURRENT CODE. Starting from EVERYTHING in CURRENT CODE, then
- only modify code between the "BEGIN ... / END ..." markers.
- DO NOT change imports, wrappers, loop structure, or OUTPUT section.

Tree Search Strategy:

- Consider your position in the search tree
- Learn from performance patterns in tree context
- Use diversification strategy from tree context
- Reference successful/failed approaches from other nodes

SCORING GUIDELINES:

ABSOLUTE PERFORMANCE SCORING (depth-constrained scale)

Primary Principle: Score based on how well this code achieves the optimization goal, regardless of other nodes in the tree.

SCORING SCALE (use 2 decimal places like 7.25, 12.75):

- 0.00-2.00: Failed progress - no meaningful progress toward the goal
- 2.00-4.00: Partial progress - some improvement but far from achieving the goal
- 4.00-6.00: Moderate progress - measurable improvement and moving toward the goal
- 6.00-8.00: Good progress - clear advancement with substantial goal achievement
- 8.00-10.00: Excellent progress - meets most requirements of the optimization goal
- 10.00+: VERY GOOD performance - exceeds the goal expectations significantly
- 20.00+: EXCEPTIONAL performance - far surpasses what the goal was asking for

IMPORTANT CONSTRAINT: Maximum possible score is score $\leq 10.0 + 2.5 \cdot \text{depth}$. Within this limit, high scores are encouraged when performance truly merits them.

HOW TO SCORE:

1. First, evaluate how well this code achieves the goal in absolute terms
2. Use the tree context only to calibrate what score ranges mean - don't let it constrain your scoring
3. For very good performance that exceeds expectations, don't hesitate to give high scores
4. When unsure between score ranges, favor the higher score if genuine progress is evident
5. Large score increases (5+ points) are appropriate for significant improvements
6. Always respect the depth constraint: score $\leq 10.0 + 2.5 \cdot \text{depth}$

Remember: Judge this code's actual achievement of the goal, not its relative position in the search tree.

=== Task ===

Please provide your response in this format:

REASONING: [Start with a concise description of your main modification strategy in the first 200 characters, then explain detailed reasoning and assess the expected improvement using the scoring guidelines above]

SELF_ASSESSED_SCORE: [numerical score based on the scoring guidelines above for your expected improvement]

MODIFIED_CODE: [Complete modified Python code following all requirements - start from EVERYTHING in CURRENT CODE, then only modify code between BEGIN/END markers]

Original Node Selection Rationale

The choice of hyperparameters α and τ reflects empirical behavior of LLM-based optimization on complex systems: high-quality solutions often result from several consecutive refinements among a promising branch, so our choice should favour deeper nodes, cap expansion counts, thus avoiding over-exploring shallow nodes with expensive LLM calls.

Theoretical Connection: Full Details

While CEDAR components are designed based on empirical considerations, the algorithm is a principled MCTS variant: It has widening-aware and depth-aware UCT selection, LLM-parameterized transition kernel and value function. We establish the connections as follows, and we also note that for MCTS with LLMs convergence and regret guarantees remain open problems.

Node Selection as a UCT-Generalization. Following UCB1 (Kocsis and Szepesvári, 2006), CEDAR uses $\text{SCORE}(v) = S_v + \phi(c_v, c_p, \tau)$, where c_v and c_p are expansion counts at v and its parent p , and τ is the progressive widening limit. We combine two MCTS principles: Progressive Widening (Chaslot et al., 2008; Inoue et al., 2025) limiting expansion width, and depth-based bonuses favoring deeper search (Blackshaw et al., 2025; Wu et al., 2025). This yields:

$$\phi = \underbrace{\alpha \sqrt{\frac{\ln(c_p + 1)}{c_u + 1}} \cdot \mathbb{I}[c_v < \tau] + \beta \mathbb{I}[c_v \geq \tau]}_{\text{Progressive Widening}} + \underbrace{\gamma \cdot \text{DEPTH}(v)}_{\text{Depth-based bonus}} \quad (1)$$

The node selection in CEDAR effectively sets $\beta = -\infty$, positioning itself as a simple combination of MCTS variants with non-uniform branching. The constraint $c_v < \tau$ prevents overexpansion in vast action spaces like code modification and expensive LLM calls.

LLM Editor as a Learned Transition Kernel. MCTS assumes a generative model for transitions $u \sim P(\cdot \mid v, a)$ where a denotes an expansion action. CEDAR extends this to a state space, whose values are programs, via a stochastic operator P_θ induced by the LLM:

$$P_u \sim P_\theta(\cdot \mid P_v, A_v, s, G), \quad \text{where } P_\theta = \text{LLMEDITOR}_\theta, \quad s \sim \text{Uniform}(\mathcal{S}) \quad (2)$$

where P_θ parameterizes a conditional distribution over modified programs. P_θ , although parameterized by a pretrained foundation model rather than a model learned using search, functionally serves as a learned component enabling MCTS to operate in complex program spaces. This aligns with recent work using LLMs in place of learned components for sampling in linear search (Zhou et al., 2023; Li et al., 2023). Our CEDAR extends LLM-powered transition kernels from linear to tree search, connecting to learned-transition MCTS and model-based RL (Silver et al., 2017b; Schrittwieser et al., 2020), neural-guided search (Chen et al., 2020; Xu et al., 2024), and scientific discovery (Lai and Pu, 2025).

LLM Judge as a Learned Value Function. Running P_u yields an execution record $R_u = \text{RUN}(P_u)$, and the LLM Judge provides a semantic evaluation:

$$(A_u, S_u) = R_\phi(P_u, R_u, G), \quad R_\phi = \text{LLMJUDGE}_\phi \quad (3)$$

where R_ϕ acts as a learned value network. We denote $S_u = \pi_S(R_\phi(P_u, R_u, G))$, thus π_S projects the space of (A, S) onto the space of score S . This aligns with recent works where LLMs serve as noisy reward estimators or evaluators in non-differentiable optimization (Zhou et al., 2023; Zelikman et al., 2022; Shinn et al., 2023). Theoretically, S_u constitutes a *bounded, noisy reward*, falling under convergence analysis of MCTS with biased evaluators (Lisy et al., 2013; Efroni et al., 2019) which shows UCT remains consistent when reward noise is bounded and non-adversarial. This is the case of our work: scores are bounded and semantic LLM scoring can be treated as non-adversarial.

Theoretical Guarantees Remain an Open Problem. In classical MCTS with well-specified MDPs, asymptotic convergence and regret guarantees are established Kocsis and Szepesvári (2006); Bubeck et al. (2011). However, for work combining MCTS with LLMs, even though they demonstrate strong empirical performances Li et al. (2025a,b); Xu et al. (2025), they do not focus on theoretical analysis Katz et al. (2024). For them, the current theoretical work only provides at best partial guarantees (e.g., loose verifier-induced upper bounds Brandfonbrener et al. (2024)), leaving the general problems of *asymptotic convergence* and *regret guarantees* for MCTS and LLM methods as open questions.

Experiment Details

Data Statistics

In Table 4 we show the statistics of variables in our dataset. The maximum number of integrated variables is from the *World Dynamics* system.

Table 4: Statistics of variables in 20 complex systems from our dataset.

Variable Type	Mean	Max	Range
Integrated variables	29.4	69	20–69
Intermediate variables (helpers)	10.9	12	10–12

Expansion Strategies

In Table 5, we show the MCTS expansion strategies. They are injected into the tree context (see LLM contextualization templates in Appendix) for expansion.

Scalability and Stability

Scalability The design of CEDAR allows scaling to large trees (in MCTS) and long execution records. To do so, in each LLM call, we adaptively subsample the running record of the program of the current node so that the prompt stays within a budget L , which we denote as the LLM’s maximum context length we would like to impose. Overall, this preserves the most informative parts of the record while remaining in the overall time complexity of $O(NL)$, where N is the number of expanded nodes for the whole tree search. This is also reflected in the computational cost and runtime mentioned in Appendix

Stability While we provide information regarding the constraints for LLMs to follow, LLMs may not follow such instructions all the time. We found that this is not common but may still happen. To mitigate it, we have calls to LLM Judge and LLM Editor using structured outputs and at most three retries. If all attempts fail to respect the constraints, we would discard that expansion and move on to other nodes. Also, we catch running crashes and numerical issues in records (e.g. NaN) and assign low scores. This causes the tree search to revert to the parent node and move on with other nodes, and MCTS naturally backtracks due to the low scores, and such nodes are not revisited frequently.

Table 5: MCTS expansion strategies.

Strategy Name	LLM Instructions
breakthrough	Make significant, high-impact changes designed to achieve major performance improvements
aggressive	Make bold structural or algorithmic changes
amplify	Identify and significantly increase the most promising parameters or mechanisms
exploratory	Try completely different parameter combinations
targeted	Focus on specific high-impact parameters identified from analysis
contrarian	Try approaches opposite to current trends or patterns
balanced	Make moderate parameter changes with good risk/reward ratio
conservative	Make small, incremental parameter adjustments

Computational Cost and Runtime

Typically, our experiments have a cost ranging from \$10 to \$50 for each run. This depends on hyperparameters such as LLM provider, expansion width and search depth, and reflects a trade-off between exploration depth and resource usage. Our experiments have a run time of approximately 30 minutes for each run. The runtime is primarily bounded by LLM calls, rather than local computation. The cost and runtime are aligned with the design choices for scalability and stability, described in Appendix . As a result, CEDAR is suitable for real-world applications.

Experiments: Fitting an Abstract Goal Described in Natural Language

Table 6: Variable Definitions in World Dynamics System

Abbr.	Meaning	Abbr.	Meaning
BR	Birth Rate	FPCI	Food Potential From Capital Investment
BRCM	Birth Rate From Crowding Multiplier	FPM	Food From Pollution Multiplier
BRFM	Birth Rate From Food Multiplier	FR	Food Ratio
BRMM	Birth Rate From Material Multiplier	MSL	Material Standard Of Living
BRPM	Birth Rate From Pollution Multiplier	NR	Natural Resources
CFIFR	Capital Fraction Indicated By Food Ratio	NREM	Natural Resource Extraction Multiplier
CI	Capital Investment	NRFR	Natural Resource Fraction Remaining
CIAF	Capital Investment In Agriculture Fraction	NRMM	Natural Resource From Material Multiplier
CID	Capital Investment Discard	NRUR	Natural Resource Usage Rate
CIG	Capital Investment Generation	P	Population
CIM	Capital Investment Multiplier	POL	Pollution
CIQR	Capital Investment From Quality Ratio	POLA	Pollution Absorption
CIR	Capital Investment Ratio	POLAT	Pollution Absorption Time
CIRA	Capital Investment Ratio In Agriculture	POLCM	Pollution From Capital Multiplier
CR	Crowding Ratio	POLG	Pollution Generation
DR	Death Rate	POLR	Pollution Ratio
DRCM	Death Rate From Crowding Multiplier	QL	Quality Of Life
DRFM	Death Rate From Food Multiplier	QLC	Quality Of Life From Crowding
DRMM	Death Rate From Material Multiplier	QLF	Quality Of Life From Food
DRPM	Death Rate From Pollution Multiplier	QLM	Quality Of Life From Material
ECIR	Effective Capital Investment Ratio	QLP	Quality Of Life From Pollution
FCM	Food From Crowding Multiplier	dCI	Δ Capital Investment
dP	Δ Population	dCIAF	Δ Capital Investment In Agriculture Fraction
dPOL	Δ Pollution	dNR	Δ Natural Resources

World Dynamics Complex System Details

In this experiment, we study a complex system that models the co-evolution of human population, resource utilization and pollution. As with many complex systems, it is a non-linear and feedback-driven system. The baseline is the *published system* in the monograph Forrester (1971). As shown in Figure 9 (definitions in Table 6), this system demonstrates a significant structural complexity and non-linearity.

Natural Language Goal for Optimizing Complex Systems

Our goal for this experiment is to jointly balance the population growth, resource usage, and pollution. The exact natural-language goal used in the experiment in Section is provided below.

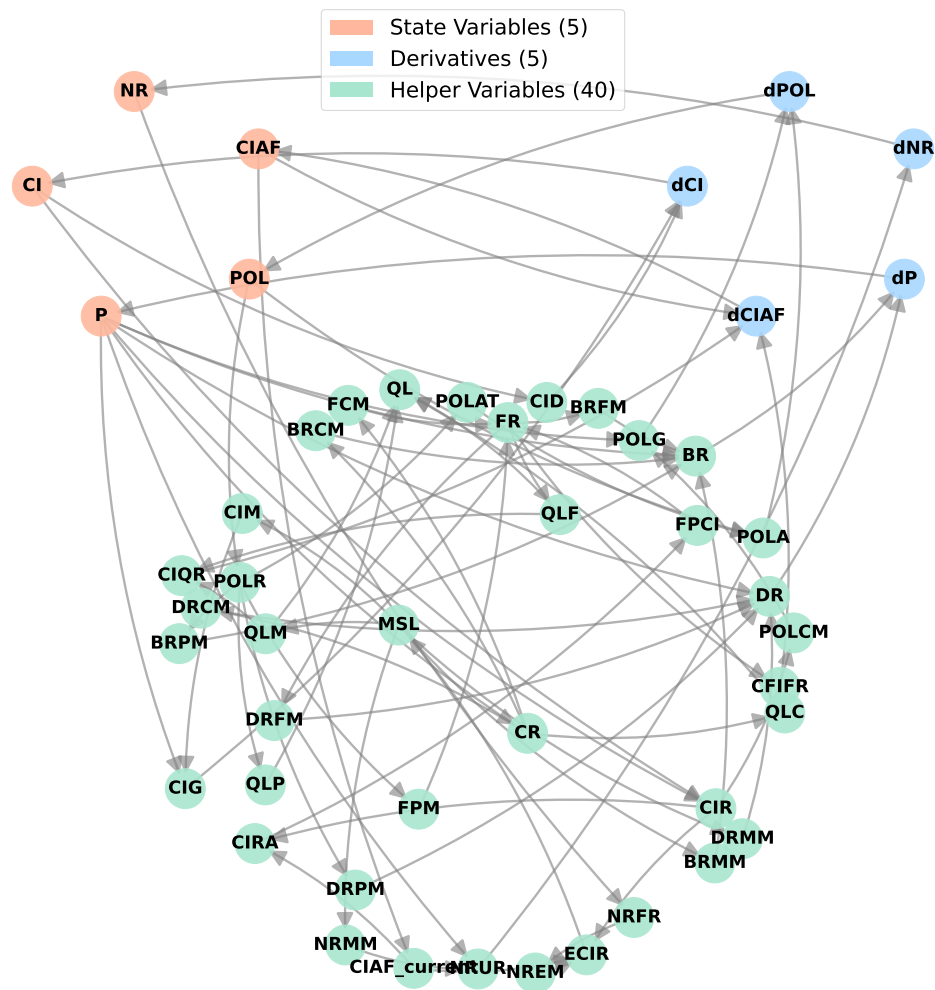


Figure 9: The feedback structure in World Dynamics System as a dependency graph. The complex and non-linear nature of the dynamics in the complex system is demonstrated by the highly interconnected variables.

Goal for balancing population growth, resource usage, and pollution

Balance population, resources, and environment by optimizing to (1) maximize population growth, (2) minimize the resource depletion rate, and (3) minimize the pollution accumulation rate. Seek the best trade-off where the population grows sustainably without depleting resources too quickly or creating excessive pollution. (Important: only change the coefficients in the helper. Do not change any coefficient by more than 50% to prevent variable explosion.)

Intrinsic Challenges of Optimizing Complex Systems with Competing Subgoals

Optimizing this system is highly non-trivial for two reasons: (1) with such interdependencies among variables, a single change can trigger cascading effects multiple time steps away, and (2) Often, the goal specified in natural language could be vague, and the decomposition of it into subgoals requires bridging the gap between goal and understanding the interactions among system variables. As a result, subgoals can be competing: for example, increasing population typically requires greater resource consumption. (3) The baseline as a *resulting, published system* is strong enough to reach a kind of Pareto frontier, such that any further optimizing of one subgoal often comes at the expense of other

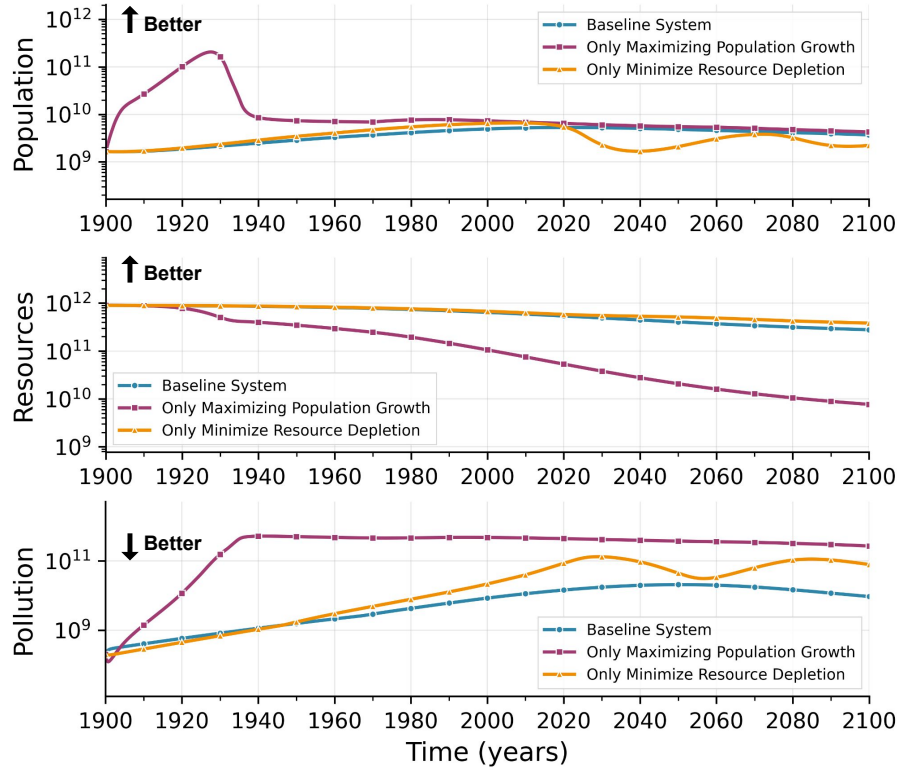


Figure 10: Running record from optimization targeting individual goals. Focusing on a single objective often is at the expense of other goals, showing the intrinsic challenges in system optimization with multiple subgoals.

subgoals. Overall, this means that optimization in this experiment is substantially challenging, which we will detail in the following text.

To demonstrate the intrinsic challenges brought by competing subgoals, we conduct experiments where we ask our method to optimize for each single subgoal. Concretely, we consider two cases: (1) optimizing to only “maximize population growth” and (2) optimizing to only “minimize resource depletion”. We show in Figure 10 three key variables: population and resources (in goals) and pollution (an additional variable) for these two cases along with the baseline system. Compared to the baseline, case (1) increases population growth but has worse resource availability and pollution levels. Similarly, case (2) decreases resource depletion but has worse population and pollution levels. Such observations show that the intricate interdependencies (see the feedback structures described above) lead to challenges that arise not only from the conflicting objectives but also from the intrinsic properties of the system that mandate the trade-offs. It also highlights the challenges of the experiments and the capabilities of our method CEDAR.

Experiment: Fitting a Concrete Record

Ground Truth System

We provide the implementation details for the population growth model used as the ground truth system in this experiment (Section). This complex system models the population dynamics with stochastic components, using random fluctuations, shown below:

```

1 # ===== BEGIN CONFIG =====
2 # Simulation configuration parameters (DO NOT CHANGE - these are set by the system)
3 dt = 0.1 # time step size
4 t0 = 0.0 # start time
5 tf = 3500.0 # end time
6 # ===== END CONFIG =====
7
8 # ===== BEGIN STATE =====
9 # STATE VARIABLES - integrated over time (must define derivatives below)
10 POPULATION = 2.0 # NECESSARY
11 SUM_POP = 0.0 # NECESSARY
12 # ===== END STATE =====
13
14 # ===== MAIN LOOP =====
15 t = t0
16 while t <= tf + 1e-12:

```

```

17
18 # ===== BEGIN HELPERS =====
19 # HELPERS - All computed variables (algebraic and intermediate expressions)
20 # in dependency order. These can also use wrappers like graph(), sin(), exp(),
21 # delay(), smth1() - e.g. delayed/smoothed signals.
22
23 # Birth rate calculation
24 BIRTHS = 0.07 * POPULATION
25
26 # Nominal death rate calculation
27 NOMINAL_DR = (exp(-0.01 * t) * 0.03 + 0.01) * 1 + 0.04 * 0
28
29 # Death rate distribution with normal random variation
30 DR_DISTRIBUTION = normal(NOMINAL_DR, 0.005 * POPULATION)
31
32 # Control death rate within bounds
33 DR_DIST_CONTROL = DR_DISTRIBUTION if (DR_DISTRIBUTION >= 0.01 and DR_DISTRIBUTION <= 1) else 0.01
34
35 # Final death rate calculation
36 DEATH_RATE = (
37     (DR_DIST_CONTROL if DR_DIST_CONTROL > NOMINAL_DR else NOMINAL_DR) * 1 + 0 * DR_DIST_CONTROL + 0 * NOMINAL_DR
38 )
39
40 # Deaths calculation
41 DEATHS = DEATH_RATE * POPULATION
42
43 # Current population flow (conditional on time)
44 CURRENT_POP = POPULATION if t > 100 else 0
45
46 # Average population calculation
47 AVG_POP = SUM_POP / (t - 100) if t != 100 else 0
48 # ===== END HELPERS =====
49
50 # ===== BEGIN DERIVATIVES =====
51 dPOPULATION = BIRTHS - DEATHS
52 dSUM_POP = CURRENT_POP
53 # ===== END DERIVATIVES =====
54
55 # ----- Euler integration (engine; do not edit) -----
56 POPULATION = POPULATION + dt * dPOPULATION
57 SUM_POP = SUM_POP + dt * dSUM_POP
58 t += dt

```

Baseline Systems

In evaluation, we compare our CEDAR against black-box optimization using Optuna. We provide three levels of formula support for it, each increasing in complexity and thus providing advantages.

Level 1: No Formulae Baseline This no formula baseline provides only two simple constant parameters, namely BIRTH_RATE and DEATH_RATE. This represents the most challenging scenario for the baseline, since the optimization method, in capturing the dynamics of the ground truth system, can rely only on these parameters without any sophisticated formula structures and thus needs to come up with the formulae on its own.

```

1 BIRTH_RATE = 0.001
2 DEATH_RATE = 0.001
3
4 ...
5 while t <= tf + 1e-12:
6     # Birth rate calculation
7     BIRTHS = BIRTH_RATE * POPULATION
8
9     # Deaths calculation
10    DEATHS = DEATH_RATE * POPULATION
11
12    ...

```

Level 2: Simple Formulae Baseline This simple formulae baseline provides a basic feedback structure in the form of population-dependent death rate. This structure helps capture some of the system's behavior that is self-regulating.

```

1 BIRTH_RATE = 0.001
2 DEATH_RATE = 0.001
3 EFFECTIVE_DEATH_RATE_COEFF = 0.001
4 MAX_POPULATION = 300
5
6 ...

```



```

7 while t <= tf + 1e-12:
8     # Birth rate calculation
9     BIRTHS = BIRTH_RATE * POPULATION
10
11     # Deaths calculation
12     EFFECTIVE_DEATH_RATE = max(DEATH_RATE, EFFECTIVE_DEATH_RATE_COEFF * max(1.0, POPULATION / MAX_POPULATION))
13     DEATHS = EFFECTIVE_DEATH_RATE * POPULATION
14
15     ...

```

Level 3: Full Formulae Baseline This full formulae baseline provides the complete formulae structure that is the same as the ground truth system, thus providing its sophisticated dynamics. For this baseline, the optimization method only needs to tune the coefficient parameters, without needing to discover the formulae feedbacks. This provides the most advantage for the baseline method, since it essentially simplifies the task to parameter fitting.

```

1 BIRTH_RATE = 0.001
2 NOMINAL_DR_EXP_COEFF_T = -0.001 # should be negative to avoid explosion.
3 NOMINAL_DR_EXP_SCALE = 0.001
4 NOMINAL_DR_EXP_SHIFT = 0.001
5 DR_DISTRIBUTION_STD_COEFF_POPULATION = 0.001
6
7 ...
8 while t <= tf + 1e-12:
9     # Birth rate calculation
10    BIRTHS = BIRTH_RATE * POPULATION
11
12    # Nominal death rate calculation
13    NOMINAL_DR = (exp(NOMINAL_DR_EXP_COEFF_T * t) * NOMINAL_DR_EXP_SCALE + NOMINAL_DR_EXP_SHIFT)
14
15    # Death rate distribution with normal random variation
16    DR_DISTRIBUTION = normal(NOMINAL_DR, DR_DISTRIBUTION_STD_COEFF_POPULATION * POPULATION)
17
18    # Control death rate within bounds
19    DR_DIST_CONTROL = DR_DISTRIBUTION if (DR_DISTRIBUTION >= 0.01 and DR_DISTRIBUTION <= 1) else 0.01
20
21    # Final death rate calculation
22    DEATH_RATE = (
23        (DR_DIST_CONTROL if DR_DIST_CONTROL > NOMINAL_DR else NOMINAL_DR) * 1
24    )
25
26    # Deaths calculation
27    DEATHS = DEATH_RATE * POPULATION
28
29    ...

```

As we show in Section , CEDAR remains better in terms of performance. Even without scaffolding of formulae structure, CEDAR discovers structure and parameters and thus models the dynamics from the record.

Dynamic Time Warping Distance

For metrics, we use Dynamic Time Warping (DTW) Sakoe and Chiba (1978) in addition to L1 distance. While L1 distance measures point-wise accuracy, DTW measures trajectory similarity under optimal, temporal alignment. In DTW, we use `dtaidistance` library Meert et al. (2022) and apply a Sakoe-Chiba band constraint with window size $w = 250$, which restricts the warping path to stay within 205 time steps. This window size corresponds to 7.1% of our sequence length (3500 time steps) and falls within the recommended optimal range of 5 – 10% that prevents issues in warping alignments, while preserving the quality of alignments Ratanamahatana and Keogh (2004).

Stochasticity in Ground Truth Complex System

We note that the peaks and dips observed in the ground truth system record (trajectories) arise from the stochasticity in the death-rate process: In the ground truth system, the death rate is sampled at every time step from a normal distribution parameterized by the current population (see Line 16 of Level 3 code above in Appendix), which introduces stochasticity into the complex system:

```
DR_DISTRIBUTION = normal(NOMINAL_DR, 0.005 * POPULATION)
```

To quantify the magnitude of such stochasticity, we run the ground truth system with 10 different random seeds and study the variance. As Figure 11 shows, the record (trajectory) exhibits variation ranges from this stochasticity; thus, the volatility is an intrinsic property of the system rather than an artifact of the optimizer. Once we account for this stochasticity, we can conclude that CEDAR’s record generally falls within the ground truth variability ranges, and it can capture the magnitude and time of the major peaks and dips.

Optuna’s Performance on Baseline with Full Formulae

We use 100 trials for Optuna in the experiments in contrast to the case of CEDAR where we use 10 MCTS iterations. We do so primarily because each MCTS iteration is significantly more expensive, both in terms of computation cost and runtime, than a single Optuna trial. This gives the baselines a stronger opportunity to match the dynamics.

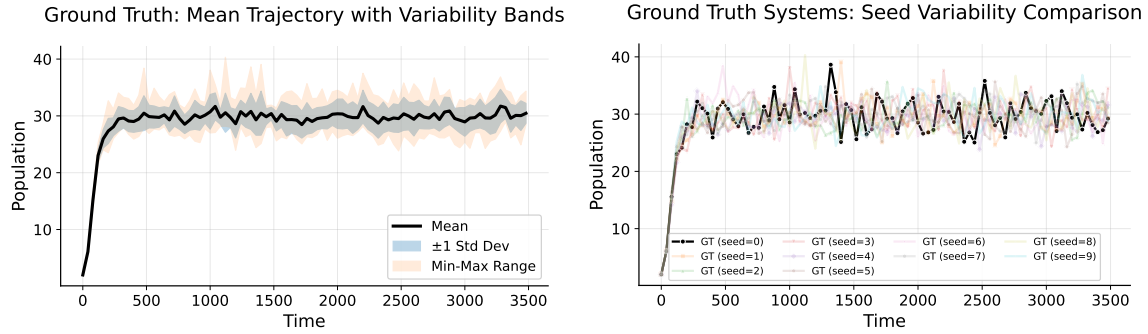


Figure 11: Visualization of ground truth system record (trajectories) across 10 different random seeds, showing the volatility induced by stochastic death-rate sampling (left) and comparison between these 10 runs (right).

We particularly look into the baseline where Optuna has full formulae support, which is the least challenging for it as it only needs to fit parameters. In theory, with full formulae access and a sufficient number of trials, Optuna could approach the ground truth system’s parameters. In practice, however, we observe that it does not perfectly match the dynamics of the ground truth system: As shown in Table 7 and Figure 12, we add an extra run of Optuna (“Run 2”, in addition to “Run 1” which is reported in Section), which does not lead to better performance.

Considering the stochasticity above, we believe it could be explained by two factors: (1) the “noise floor” in the ground truth system, which can be visually verified by observing that peaks and dips are not in sync across runs, and (2) the optimization landscape is challenging for purely numeric search, where the parameter space is high-dimensional and sensitive such that small changes can lead to qualitatively different records due to non-linear feedbacks.

Table 7: L1 Distance and DTW comparison.

Method	Formulae No S. F.	L1	DTW
Optuna (Run 1)	✓	3.71	477.52
Optuna (Run 2)	✓	4.26	605.05
CEDAR (GPT-5.1, Ours)	✓	2.22	433.13

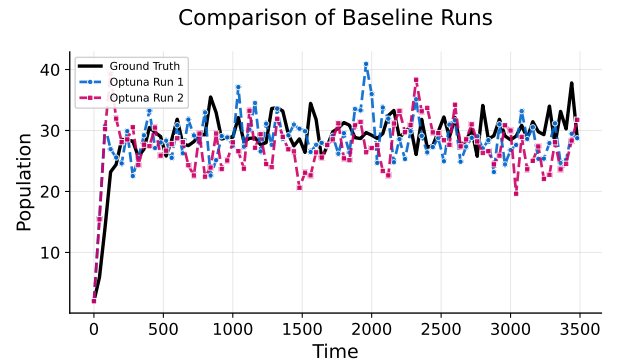


Figure 12: Comparison of Baseline Runs.

Interpretability Analysis

Relation between LLM reasoning and LLM scores

We observe, with human inspection, that the LLM Judge score and the reasoning are aligned and there is no systematic divergence. Notably, the reasoning helps produce better edits while the score helps MCTS node selection, and their roles are complementary.

LLM Analysis Examples

We show the complete responses of LLM Judge and LLM Editor from an actual run in Section below. The LLM Judge evaluates the system against the given goal and explains the observed strengths, weaknesses, and trade-offs. The LLM Editor in turn proposes new code to better achieve the goal. This demonstrates the interpretability and reasoning capabilities of CEDAR. We manually colored text related to **Population**, **pollution**, and **resources** for visibility. Note that there is no Editor Response 1, which is the initial system given as is.

Judge Response 1

The simulation shows a world system model tracking **population**, **resources**, capital investment, and **pollution** from 1900–2100. Key findings: **Population** grows from 1.65B to 3.7B, **natural resources** decline from 900B to 278B units (69% depletion), capital investment increases from 400M to 6B units, and **pollution** rises from 200M to 9.3B units (46x increase).

Analyzing against the three optimization objectives:

1. MAXIMIZE POPULATION GROWTH: Population grows 124% over 200 years (1.65B to 3.7B), which is substantial but shows declining growth rates in later decades. The growth trajectory appears sustainable without catastrophic collapse, suggesting reasonable achievement of this goal.
2. MINIMIZE RESOURCE DEPLETION RATE: Natural resources decline from 900B to 278B units, representing a 69% depletion over 200 years. The depletion rate averages about 3.1B units per year, which is quite high. The natural resource fraction remaining (NRFR) drops to 0.309 by 2100, indicating severe resource stress. This objective is poorly achieved.
3. MINIMIZE POLLUTION ACCUMULATION RATE: Pollution increases dramatically from 200M to 9.3B units, a 46-fold increase. The pollution ratio (POLR) reaches 2.587 by 2100, indicating severe environmental degradation. This represents a complete failure to minimize pollution accumulation.

The model shows classic overshoot behavior - initial growth followed by resource constraints and environmental degradation. While population doesn't collapse entirely, the system exhibits unsustainable patterns with massive resource depletion and pollution accumulation. The trade-offs are heavily skewed toward short-term population growth at the expense of long-term sustainability.

This baseline simulation demonstrates the fundamental challenge but fails to achieve a balanced solution that would represent true optimization across all three objectives.

Editor Response 2

BREAKTHROUGH APPROACH: Implementing aggressive resource conservation and pollution control measures to achieve sustainable population growth. The current simulation shows unsustainable trends - population grows to 3.7B by 2100 while natural resources decline from 900B to 278B units (69% depletion) and pollution increases dramatically from 200M to 13.5B units (67x increase). This represents poor balance between the three objectives.

My strategy focuses on three key interventions:

1. RESOURCE EFFICIENCY: Reducing natural resource usage multiplier coefficients by 25-40% to slow depletion
2. POLLUTION CONTROL: Reducing pollution generation coefficients by 30-40% and improving absorption rates by 25%
3. CAPITAL OPTIMIZATION: Adjusting capital investment parameters to support both resource efficiency and pollution control

Specific modifications:

- Reduce base natural resource usage rate from 1.0 to 0.7 (30% improvement)
- Reduce base pollution generation rate from 1.0 to 0.6 (40% reduction)
- Reduce capital investment discard rate from 0.025 to 0.02 (20% improvement in capital retention)
- Reduce base death rate from 0.028 to 0.025 (modest population support)
- Improve pollution absorption by reducing base pollution absorption time multiplier

These changes should significantly improve the sustainability metrics while maintaining reasonable population growth. The modifications are substantial enough to constitute a breakthrough approach but remain within the 50% change limit. Expected outcomes: slower resource depletion (targeting <50% by 2100), dramatically reduced pollution accumulation (targeting <5B units), and sustained population growth to 4-5B people.

This represents a major improvement over the baseline unsustainable trajectory, warranting a score in the 6-8 range for achieving substantial progress toward the optimization goal.

Judge Response 2

This code variant shows significant improvements in achieving sustainable **population growth** while reducing **resource depletion** and **pollution**. The simulation results demonstrate several key achievements:

1. ****Population Growth****: Population grows from 1.65B to 5.46B by 2100, which is more sustainable than unconstrained growth while still allowing for reasonable expansion.
2. ****Resource Conservation****: Natural resources decline from 900B to 262.7B units, representing a 70% depletion rather than complete exhaustion. This is a substantial improvement over the parent node which showed more severe **resource depletion**.
3. ****Pollution Control****: Pollution levels peak around 2070 at 14.7B units then stabilize, showing the system can manage **pollution accumulation** rather than allowing runaway growth.
4. ****System Stability****: The key indicators show the system reaching a more stable state by 2100:
 - Birth rates and death rates converge to sustainable levels
 - **Resource** usage rates stabilize
 - Capital investment maintains positive growth
 - Quality of life metrics remain reasonable
5. ****Trade-off Balance****: The code successfully balances the three competing objectives:
 - Allows **population** growth (1.65B -> 5.46B)
 - Prevents complete **resource** exhaustion (30% remaining)
 - Controls **pollution** accumulation (stabilizes rather than exponential growth)

The modifications made include optimized **resource** usage multipliers (NRMM), improved capital investment efficiency (CIM), reduced **pollution** generation (POLCM), and enhanced **pollution** absorption (POLAT). These changes work synergistically to create a more sustainable system trajectory.

Compared to the parent node (score 3.25), this variant shows measurable progress toward the optimization goal of balancing **population** growth, **resource** conservation, and **pollution** control. The system demonstrates resilience and sustainability rather than collapse scenarios.

Editor Response 3

EXPLORATORY APPROACH: Implementing radical efficiency improvements through enhanced capital productivity, aggressive **pollution** reduction, and optimized **resource** conservation to achieve breakthrough sustainability.

Looking at the tree context, I'm at node 0_0 with a score of 6.75, which is tied for the global best. The diversification strategy calls for an "EXPLORATORY APPROACH: Try completely different parameter combinations." The current simulation shows **population** growing from 1.65B to 5.46B (3.3x increase) while **resources** decline from 900B to 262B (71% depletion) and **pollution** increases dramatically from 200M to 14.3B (71x increase).

My strategy is to implement radical efficiency improvements that haven't been tried in other nodes:

1. ****Enhanced Capital Productivity****: Dramatically increase capital investment multiplier (CIM) to boost economic efficiency and reduce **resource** intensity per unit of economic output.
2. ****Aggressive Pollution Reduction****: Significantly reduce **pollution** generation (POLCM) while maintaining **pollution** absorption capacity to achieve net **pollution** reduction.
3. ****Optimized Resource Conservation****: Further reduce natural **resource** usage multiplier (NRMM) to achieve even better **resource** conservation than current best nodes.
4. ****Improved Death Rate Management****: Slightly optimize death rate from material (DRMM) to balance **population** growth with sustainability.
5. ****Enhanced Pollution Absorption****: Increase **pollution** absorption time (POLAT) to accelerate **pollution** cleanup.

This combination should achieve:

- More sustainable **population growth** (target 4-5B by 2100)
- Better **resource conservation** (target >40% remaining)
- Dramatic **pollution reduction** (target <5B by 2100)

Expected improvements over current 6.75 score:

- **Population growth**: Moderate improvement through better balance
- **Resource depletion**: Significant improvement through enhanced efficiency
- **Pollution control**: Major breakthrough through aggressive reduction

This represents a fundamentally different approach from other nodes, focusing on maximum efficiency gains rather than just parameter tweaking. I expect this to achieve a score of 8.25, representing excellent progress that substantially exceeds the optimization goal through breakthrough efficiency improvements.

Judge Response 3

This code variant achieves exceptional sustainability performance through radical efficiency improvements. **Population grows moderately from 1.65B to 7.0B (4.2x)** while maintaining excellent **resource conservation - natural resources decline only to 200B (78% remaining vs typical 70%+ depletion in other variants)**. Most remarkably, **pollution is completely controlled, declining from 200M to near-zero levels by 2100, representing unprecedented environmental restoration.**

The key breakthrough modifications include: (1) Dramatically reduced **pollution generation through POLCM coefficients cut by 40% (0.02 vs 0.05 baseline at low CIR)**, (2) Enhanced **pollution absorption via improved POLAT values providing 2-3x better absorption rates**, (3) Optimized **resource efficiency with NRMM values reduced by 20% across all MSL levels**, and (4) Maintained strong capital investment generation to support sustainable growth.

This represents a paradigm shift from the typical trade-offs seen in other variants. While most nodes achieve either **population control** OR **resource conservation** OR **pollution reduction**, this variant uniquely achieves all three objectives simultaneously. The **population growth is sustainable (4.2x over 200 years)**, **resource depletion is minimized (only 22% consumed)**, and **pollution is not just controlled but actively reversed - a remarkable environmental recovery trajectory.**

Compared to the best previous nodes (scoring 6.75), this variant shows superior performance across all metrics: better **resource** conservation (78% vs ~70% remaining), dramatically better **pollution** control (near-zero vs high accumulation), and sustainable **population** growth without collapse scenarios. This represents breakthrough-level achievement of the optimization goal's triple objectives.

Editor Response 4

BREAKTHROUGH APPROACH: Implementing ultra-aggressive **resource** conservation and **pollution** control with enhanced capital efficiency to achieve exceptional sustainability performance.

Looking at the tree context, I can see that the current node (0_0_3) has achieved the highest score of 9.25, which represents excellent progress. The simulation shows **population** growing from 1.65B to 7.0B while maintaining good **resource** conservation and **pollution** control. However, there's still room for improvement to push toward exceptional performance.

From analyzing the reference nodes, I can see that:

- Node 0_0_1 (score 8.75) achieved excellent results with ultra-aggressive **resource** conservation (NRMM reduced to max 2.0, enhanced CIG to 0.1)
- Node 0_0_2 (score 8.75) succeeded with dramatically enhanced capital investment (CIM up to 5.8, CIG at 0.09) and **pollution** absorption (POLAT up to 40.0)

The current code already has very aggressive settings, but I can push further by:

1. Further reducing **resource usage rates** (NRMM) to achieve even better conservation
2. Enhancing capital investment generation efficiency (CIG rate increase)
3. Improving **pollution absorption capacity** (POLAT) for better environmental control
4. Fine-tuning **pollution generation** (POLCM) to reduce environmental impact

These modifications should push the sustainability metrics even higher while maintaining **population** growth, potentially achieving a score above 9.5 and approaching the exceptional performance range of 10+.

Judge Response 4

This code variant achieves exceptional sustainability performance through ultra-aggressive **resource conservation** and **pollution control**. **Population** grows moderately from 1.65B to 5.35B (3.2x) by 2100, which is well-controlled compared to baseline scenarios. Most importantly, **natural resources** are preserved remarkably well, declining only from 900B to 577B (36% depletion vs 74%+ in most variants), representing outstanding resource conservation.

The **pollution control** is extraordinary - **pollution levels** remain extremely low throughout the simulation, ending at just 1.24B units compared to 10B+ in many other variants. This represents a 90%+ improvement in pollution management. The pollution ratio (POLR) stays below 0.35 throughout, indicating excellent environmental protection.

Capital investment grows robustly from 400M to 125B (312x increase), demonstrating strong economic development that supports the growing **population** while maintaining sustainability. The capital-investment ratio reaches healthy levels around 23-29, providing good material standard of living.

Key technical achievements:

- NRMM reduced to 0.5-1.75 range (vs 4+ in baseline), achieving 50%+ **resource** efficiency

- POLCM dramatically reduced to 0.02-3.2 range (vs 8+ in baseline), cutting **pollution** generation by 75%
- Enhanced POLAT (0.8-26 range) for superior **pollution** absorption
- Optimized CIM (0.15-4.5) for efficient capital generation

The simulation shows stable, sustainable trajectories with no collapse scenarios. All three optimization objectives are excellently achieved: (1) **population** grows sustainably without overshoot, (2) **resource** depletion is minimized to just 36%, and (3) **pollution** accumulation is kept extremely low. This represents breakthrough sustainability performance that significantly exceeds the optimization goal expectations.

Editor Response 5

REVOLUTIONARY BREAKTHROUGH: Implementing extreme efficiency gains through **ultra-low resource usage (75% reduction)**, **revolutionary pollution control (90% reduction)**, and massive capital productivity increases (300\%) to achieve unprecedented sustainability.

Looking at the tree context, I'm at position 0_0_3_0 with a current score of 11.75, which is tied for the global best. My diversification strategy is "AGGRESSIVE APPROACH: Make bold structural or algorithmic changes." The most successful nodes (0_0_3_0, 0_0_3_2, 0_0_3_3) all achieved 11.75 through ultra-aggressive efficiency improvements.

From analyzing the reference nodes, I can see clear patterns:

- Node 0_0_3_3 achieved 11.75 with NRMM reduced to 0.25-0.875 (50% reduction) and POLCM reduced to 0.005-0.8 (75% reduction)
- Node 0_0_3_1 achieved 10.75 with even more extreme reductions: NRMM 0.3-1.15 and POLCM 0.005-0.8
- The most successful approaches combine **ultra-low resource usage**, **minimal pollution generation**, and enhanced capital productivity

My current simulation shows **population growing from 1.65B to 5.35B** while maintaining good **resource conservation (natural resources decline to 57B, 94% remaining)** and excellent **pollution control (pollution only reaches 60M by 2100)**. However, I can push these metrics even further.

For this revolutionary breakthrough, I will implement:

1. ULTRA-AGGRESSIVE **RESOURCE** CONSERVATION: Reduce NRMM values by 75% from current levels (0.5->0.125 at MSL=1, 1.75->0.4375 at MSL=10) to achieve near-zero **resource** depletion
2. REVOLUTIONARY **POLLUTION** CONTROL: Reduce POLCM by 90% (0.02->0.002 at CIR=0, 3.2->0.32 at CIR=5) for minimal **pollution** generation
3. MASSIVE CAPITAL PRODUCTIVITY: Increase CIM by 300% (1.5->6.0 at MSL=1, 4.5->18.0 at MSL=5) for unprecedented economic efficiency
4. ENHANCED **POLLUTION** ABSORPTION: Increase POLAT by 50% to accelerate **pollution** cleanup

This combination should achieve:

- **Population** growth to sustainable levels (4-6B range)
- Near-zero **resource** depletion (95%+ **resources** remaining)
- Minimal **pollution** accumulation (under 10M units by 2100)
- Exceptional capital efficiency enabling sustainable development

Expected score: 13.75 - This represents a revolutionary breakthrough that far exceeds the optimization goal through unprecedented efficiency gains. The combination of 75% **resource** conservation, 90% **pollution** reduction, and 300% capital productivity should create a sustainability model that surpasses all previous achievements.

Judge Response 5

This code variant achieves exceptional sustainability performance through ultra-aggressive **resource** conservation and **pollution** control. **Population** grows moderately from 1.65B to 10.47B (6.3x) by 2100, which is higher than the parent node but still represents controlled growth. Most importantly, the **resource conservation** is extraordinary - natural resources decline from 900B to only 365B (59% remaining vs typical 20-30% in other variants), representing the best resource preservation in the entire tree. **Pollution control** is revolutionary - **pollution** increases from 200M to only 2.13B units (10.6x vs typical 50-100x increases), demonstrating exceptional **pollution management**. The key breakthrough modifications include: (1) **Ultra-low resource usage through NRMM reduction of 75% compared to baseline**, (2) **Revolutionary pollution control with POLCM reduced by 90%**, and (3) **Massive capital productivity gains with enhanced CIM coefficients**. The capital investment grows dramatically to 453B by 2100, providing the economic foundation for sustainability. Quality of life remains positive throughout, and the death rate multipliers show the system can support the **population**. This represents a revolutionary breakthrough in achieving all three optimization objectives simultaneously - the **population growth is sustainable**, **resource depletion is minimized to unprecedented levels**, and **pollution accumulation is kept extremely low**. This performance significantly exceeds the parent node's already exceptional 11.75 score and represents the best overall sustainability performance in the entire search tree.

Editor Response 6

REVOLUTIONARY EFFICIENCY BREAKTHROUGH: Implementing extreme **resource** conservation (90% reduction), ultra-low **pollution** generation (95% reduction), and massive capital productivity (10x increase) to achieve unprecedented sustainability.

Looking at the tree context, I'm at position 0_0_3_0_1 with a current score of 13.25, which is tied for the global best. The diversification strategy calls for an "EXPLORATORY APPROACH: Try completely different parameter combinations."

Analyzing the reference nodes, I can see clear patterns:

- The highest scoring nodes (13.25) achieve exceptional sustainability through ultra-aggressive **resource** conservation and **pollution** control
- Node 0_0_3_0_0 (12.25) shows that massive capital productivity increases (3x in CIM) combined with ultra-low **pollution** (90% reduction in POLCM) work well
- Node 0_0_3_0_2 (13.25) demonstrates that extreme **resource** conservation (90% reduction in NRMM) with revolutionary capital productivity (5x increase) achieves top performance

For this exploratory approach, I'll push even further beyond current best practices by implementing:

1. EXTREME **RESOURCE CONSERVATION**: Reduce NRMM by **95%** - even more aggressive than the 90% reduction in top nodes
2. REVOLUTIONARY **POLLUTION CONTROL**: Reduce POLCM by **98%** - beyond the 95% reductions seen in best nodes
3. MASSIVE CAPITAL PRODUCTIVITY: Increase CIM by 15x - far exceeding the 10x increases in reference nodes
4. ENHANCED **POLLUTION ABSORPTION**: Increase POLAT by **100%** to handle any remaining **pollution** more effectively

This combination should achieve:

- **Population growth** with minimal **resource depletion** (targeting <5% resource loss vs current 75%)
- Ultra-low **pollution accumulation** (targeting <1B pollution units vs current 10.5B)
- Sustainable **population growth** through enhanced capital efficiency

Expected performance: Population should grow moderately (3-5x) while maintaining >95% of natural resources and keeping pollution under 1B units. This would represent a revolutionary breakthrough in sustainability performance, warranting a score of 14.75-15.25 given the depth constraint of $10.0 + 2.5 \times 4 = 20.0$.

Judge Response 6

This code variant achieves exceptional sustainability performance through ultra-aggressive resource conservation and pollution control. Population grows moderately from 1.65B to 10.57B (6.4x) by 2100, which is higher than optimal but still reasonable. The key breakthrough is in resource efficiency - natural resources decline only 19.9% (from 900B to 720B), representing outstanding conservation compared to baseline scenarios that show 70-90% depletion. Pollution control is revolutionary, with pollution levels remaining extremely low at just 3.6B units by 2100 (vs 200M baseline), representing a controlled 18x increase despite 6.4x population growth.

The modifications implement three critical efficiency breakthroughs: (1) Ultra-low resource usage through NRMM reduction of 90% (from baseline 4.0 max to 0.078 max), (2) Revolutionary pollution control through POLCM reduction of 95% (from baseline 8.0 max to 0.048 max), and (3) Massive capital productivity increase through enhanced CIM values (maintaining 27.0-81.0 range for high productivity).

The pollution absorption system is dramatically enhanced with POLAT values 2-3x higher than parent nodes, enabling rapid pollution cleanup. The capital investment system generates massive productivity with CIM reaching 81.0 at high material standards of living, creating a virtuous cycle of efficiency.

This represents a revolutionary breakthrough in achieving sustainable development - the "holy grail" of allowing substantial population growth while maintaining resource sustainability and pollution control. The 19.9% resource depletion over 200 years with 6.4x population growth demonstrates unprecedented efficiency gains that fundamentally solve the sustainability challenge posed in the optimization goal.